

Transformers

Amit Shmidov | Tutorial 6

Largely relies on the corresponding presentations from DL4CV 2023, Weizmann Institute

Agenda

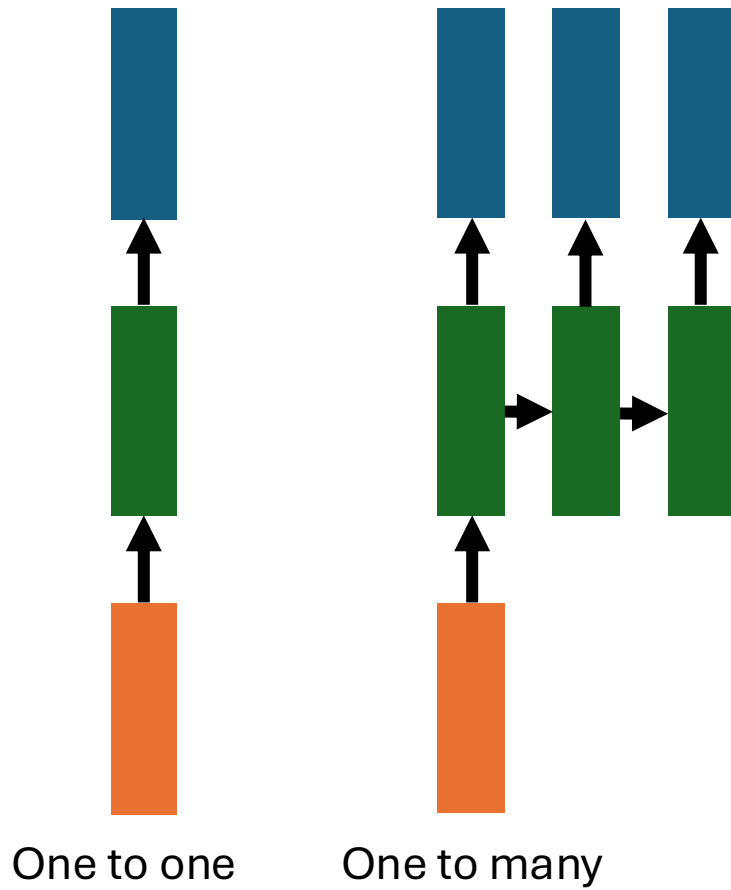
- Attention and Transformers
- Transformers in CV (ViT)

Sequences

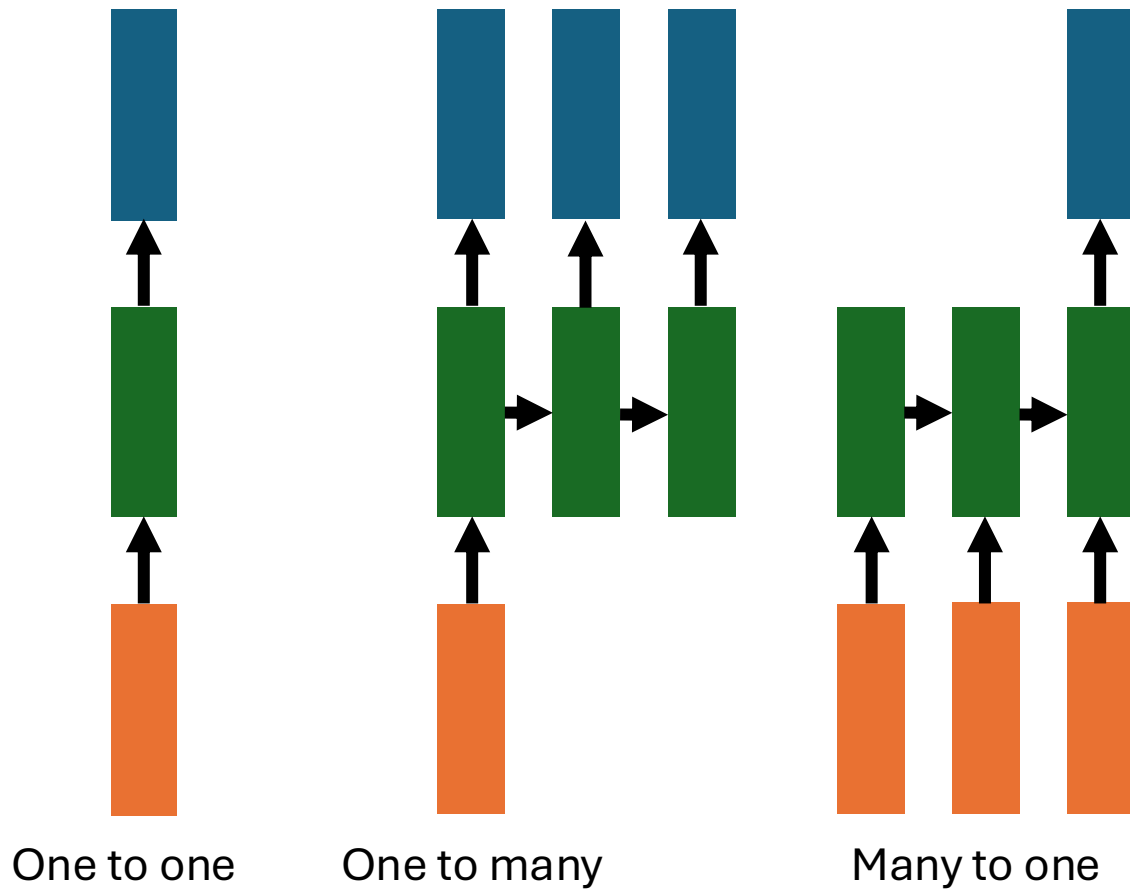


One to one

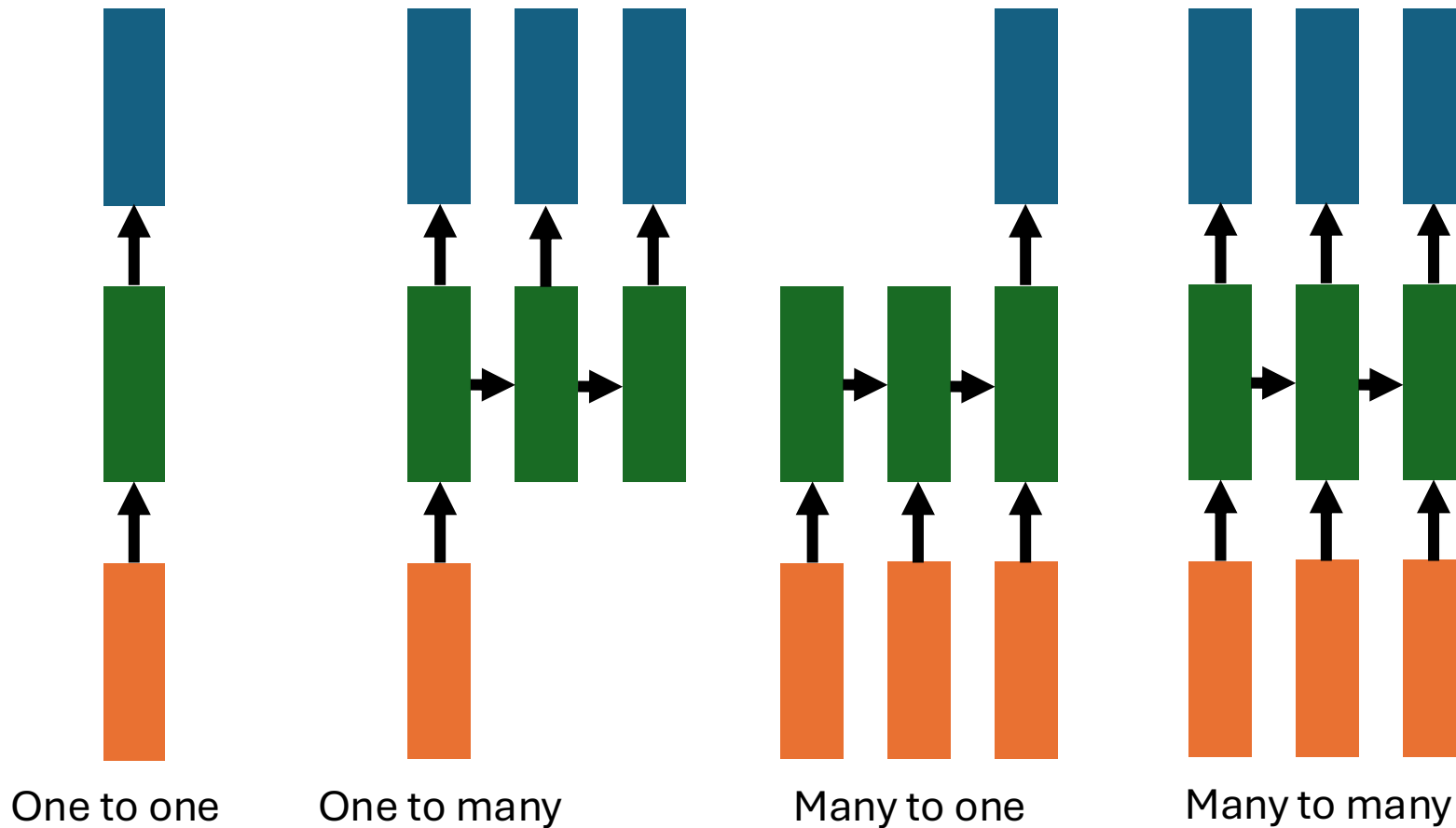
Sequences



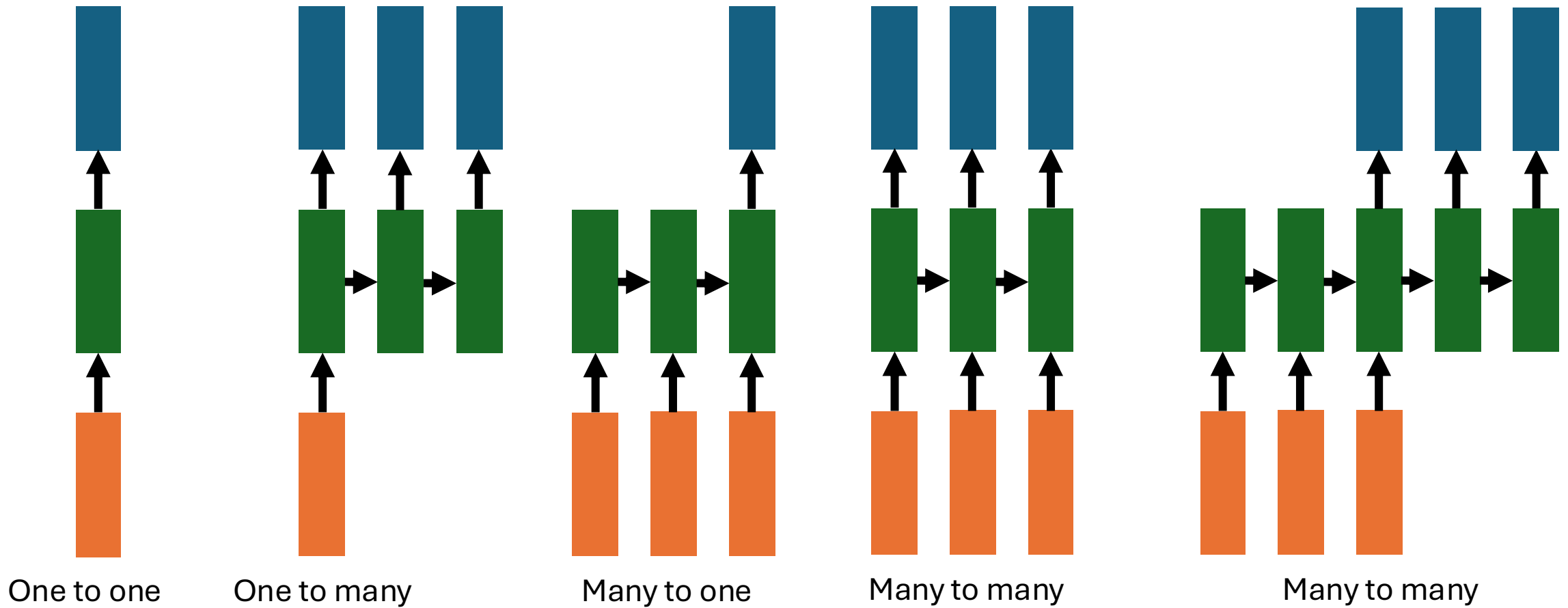
Sequences



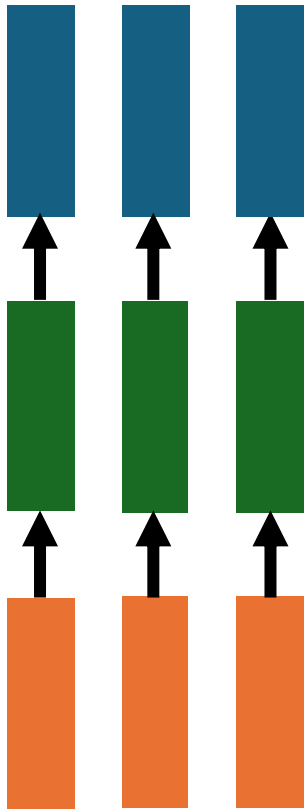
Sequences



Sequences



Recurrent neural networks



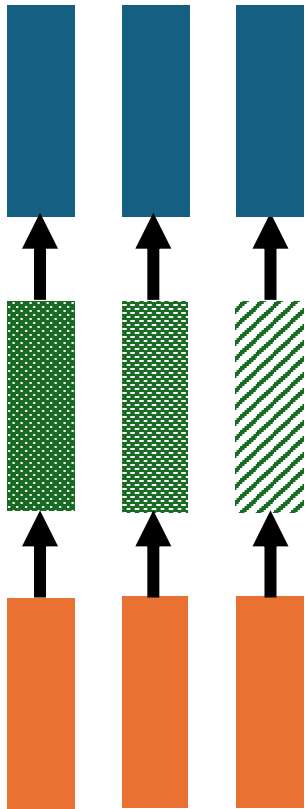
Many to many

```
class ManyToManyV0(nn.Module):
    def __init__(self, number_of_time_steps):
        super(ManyToMany, self).__init__()
        # SAME instance of SingleTimeStep for each time step
        self.time_steps = SingleTimeStep(...)

    def forward(self, in_seq):
        out_pred = []
        for t, x_t in enumerate(in_seq):
            p_t = self.time_steps(x_t)
            out_pred.append(p_t)
        return out_pred
```

- + Parameter efficiency
- No time dependency

Recurrent neural networks



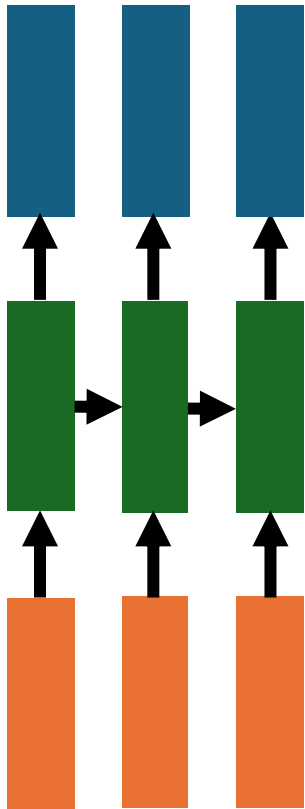
Many to many

```
class ManyToManyV1(nn.Module):
    def __init__(self, number_of_time_steps):
        super(ManyToMany, self).__init__()
        # DIFFERENT instance of SingleTimeStep for each time step
        self.time_steps = nn.ModuleList([SingleTimeStep(...)
                                           for _ in range(number_of_time_steps)])

    def forward(self, in_seq):
        out_pred = []
        for t, x_t in enumerate(in_seq):
            p_t = self.time_steps[t](x_t)
            out_pred.append(p_t)
        return out_pred
```

- + Temporal dependency
- Parameter inefficiency
- Fixed sequence length

Recurrent neural networks



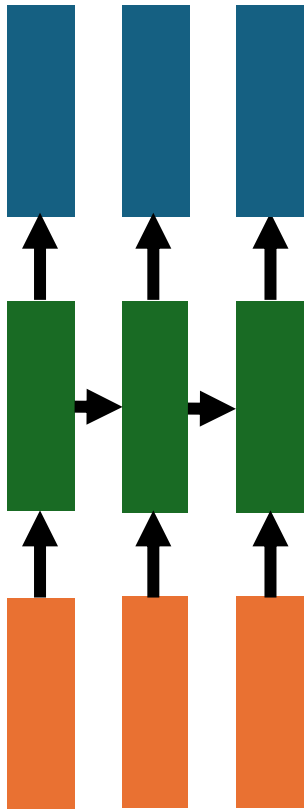
Many to many

```
class ManyToMany(nn.Module):
    def __init__(self):
        super(ManyToMany, self).__init__()
        # SHARE a single instance of SingleTimeStep
        self.time_step = SingleTimeStep(...)

    def forward(self, in_seq):
        out_pred = []
        state = self.init_state
        for t, x_t in enumerate(in_seq):
            p_t, state = self.time_step(x_t, state)
            out_pred.append(p_t)
        return out_pred
```

- + Temporal dependency
- + Parameter efficiency
- + Arbitrary sequence length

Recurrent neural networks



Many to many

```
class ManyToMany(nn.Module):
    def __init__(self):
        super(ManyToMany, self).__init__()
        # SHARE a single instance of SingleTimeStep
        self.time_step = SingleTimeStep(...)

    def forward(self, in_seq):
        out_pred = []
        state = self.init_state
        for t, x_t in enumerate(in_seq):
            p_t, state = self.time_step(x_t, state)
            out_pred.append(p_t)
        return out_pred
```

$$h_t = f(h_{t-1}, x_t; W)$$

Language modeling – training

Given $c_1, c_2, \dots, c_{t-1}$ predict c_t :

Training sequence: “hello”

Vocabulary: [h, e, l, o]

1	0	0	0
0	1	0	0
0	0	1	1
0	0	0	0
h	e	l	l

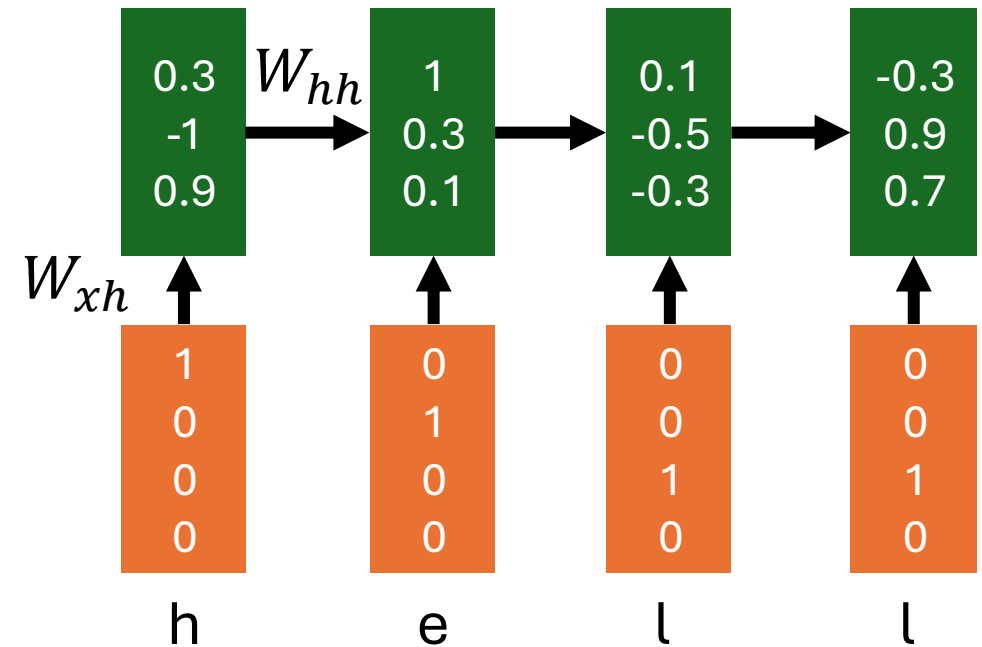
Language modeling – training

Given $c_1, c_2, \dots, c_{t-1}$ predict c_t :

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

Training sequence: “hello”

Vocabulary: [h, e, l, o]



Language modeling – training

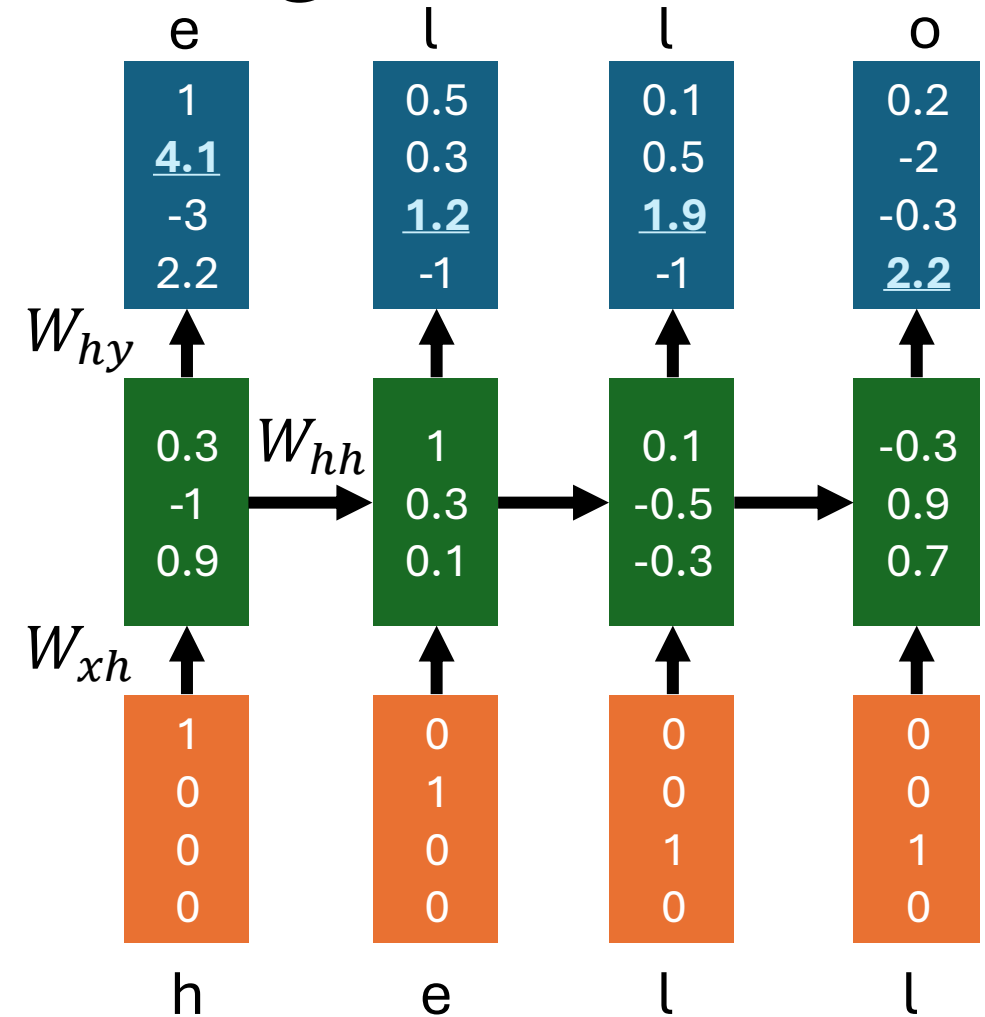
Given $c_1, c_2, \dots, c_{t-1}$ predict c_t :

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t$$

Training sequence: “hello”

Vocabulary: [h, e, l, o]



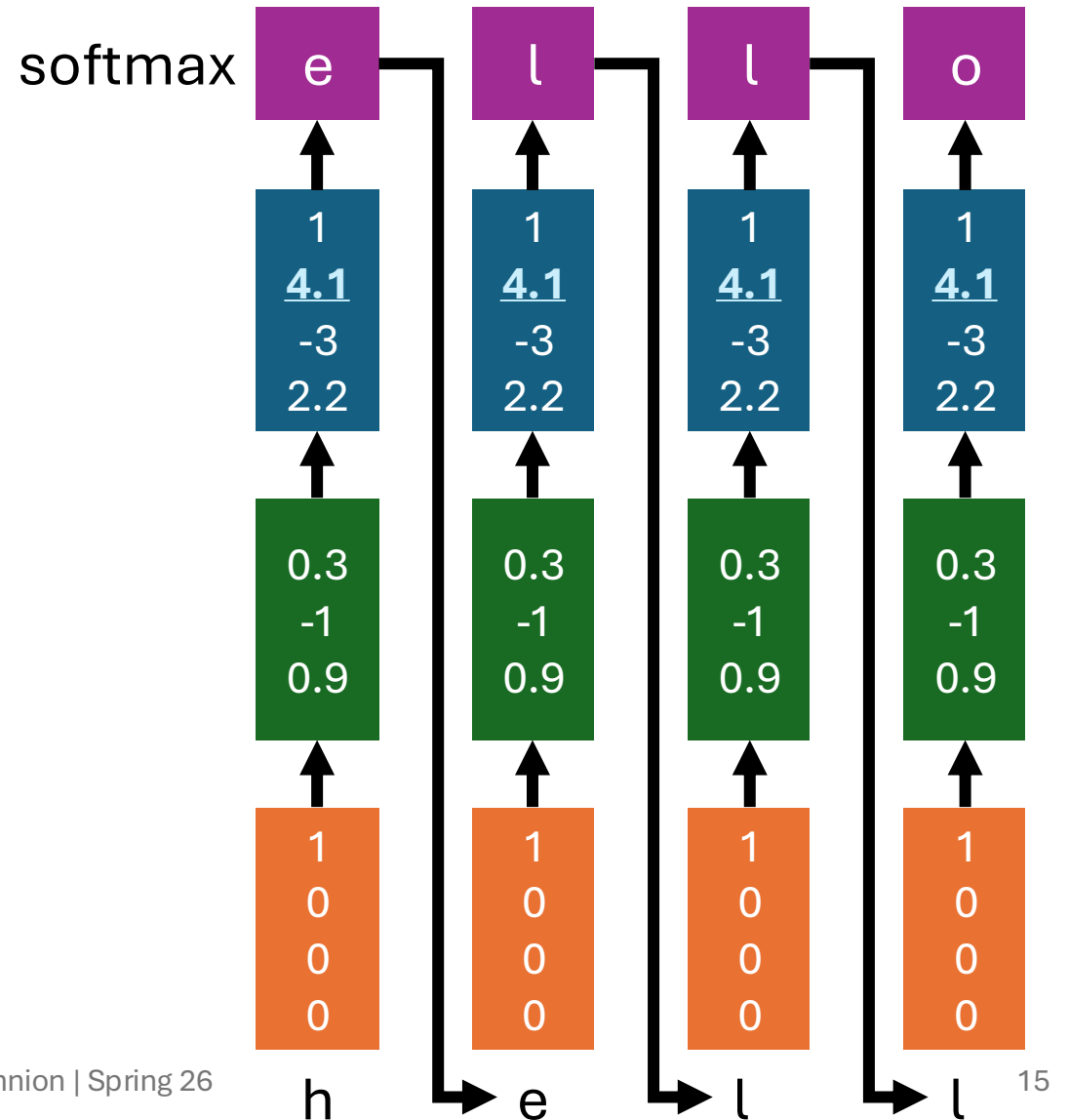
Language modeling – inference

Given $c_1, c_2, \dots, c_{t-1}$ predict c_t :

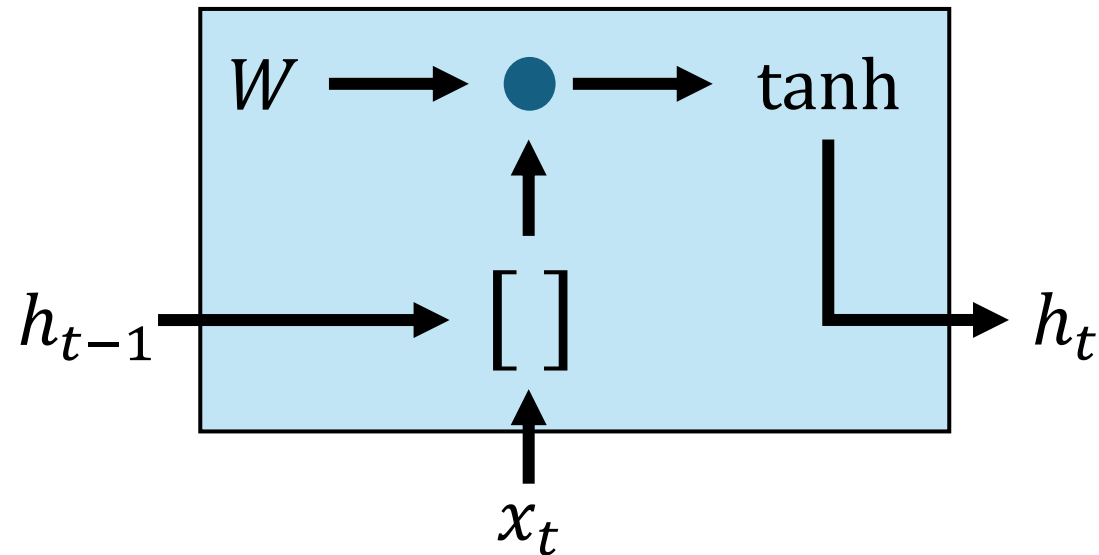
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t$$

Vocabulary: [h, e, l, o]

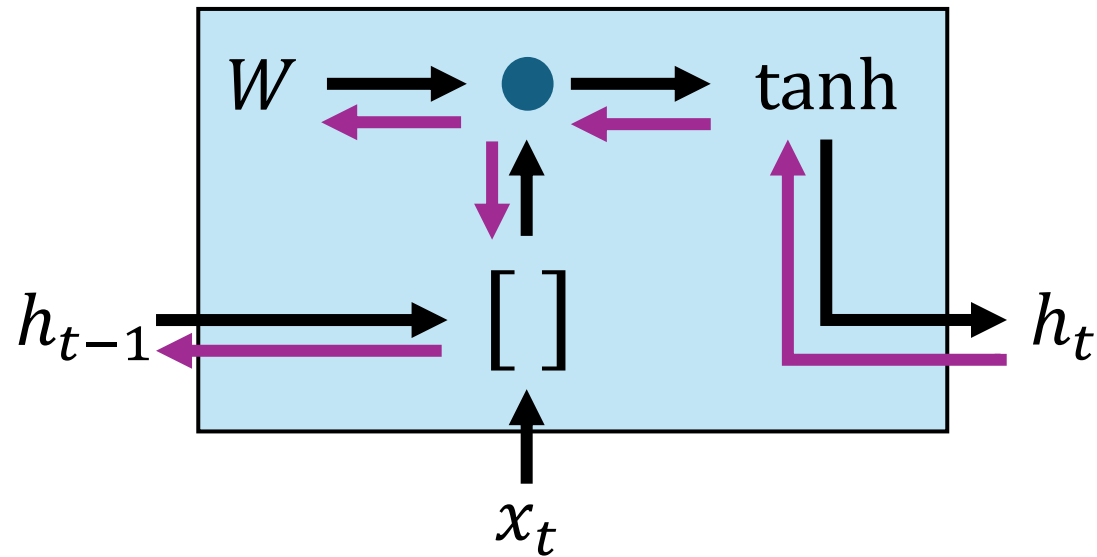


RNN – gradient flow



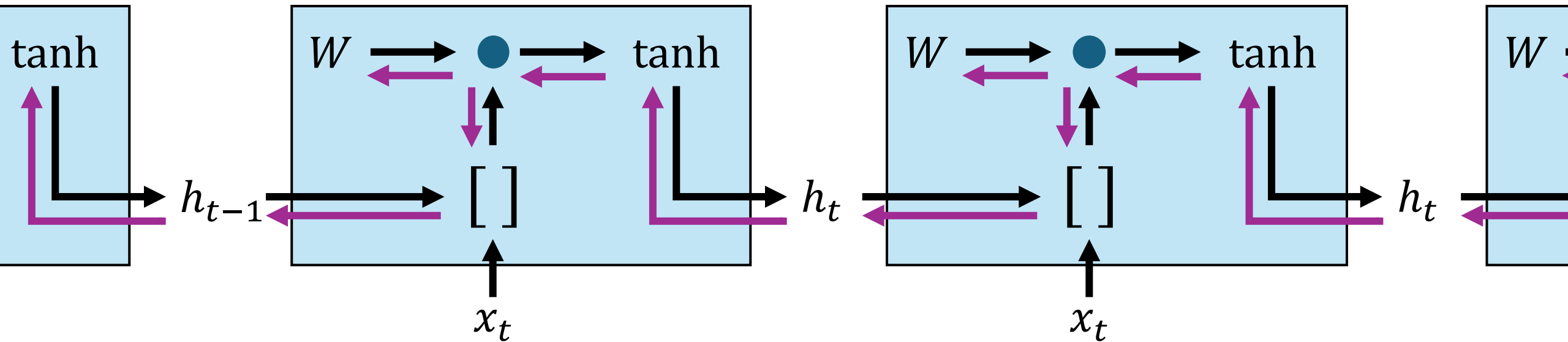
$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{xh}x_t) \\ &= \tanh\left(W \cdot \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \end{aligned}$$

RNN – gradient flow



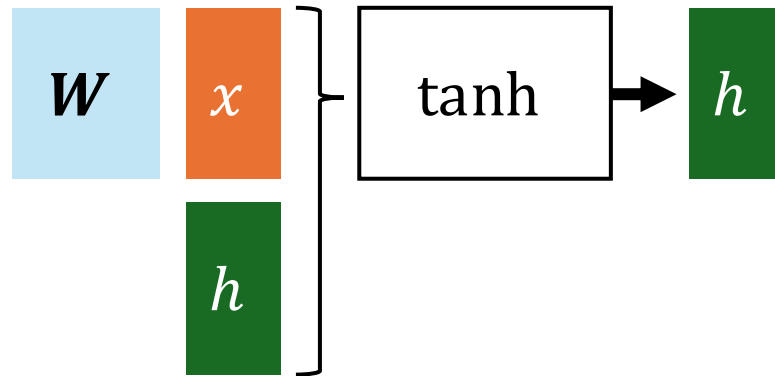
$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{xh}x_t) \\ &= \tanh\left(W \cdot \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \end{aligned}$$

RNN – gradient flow

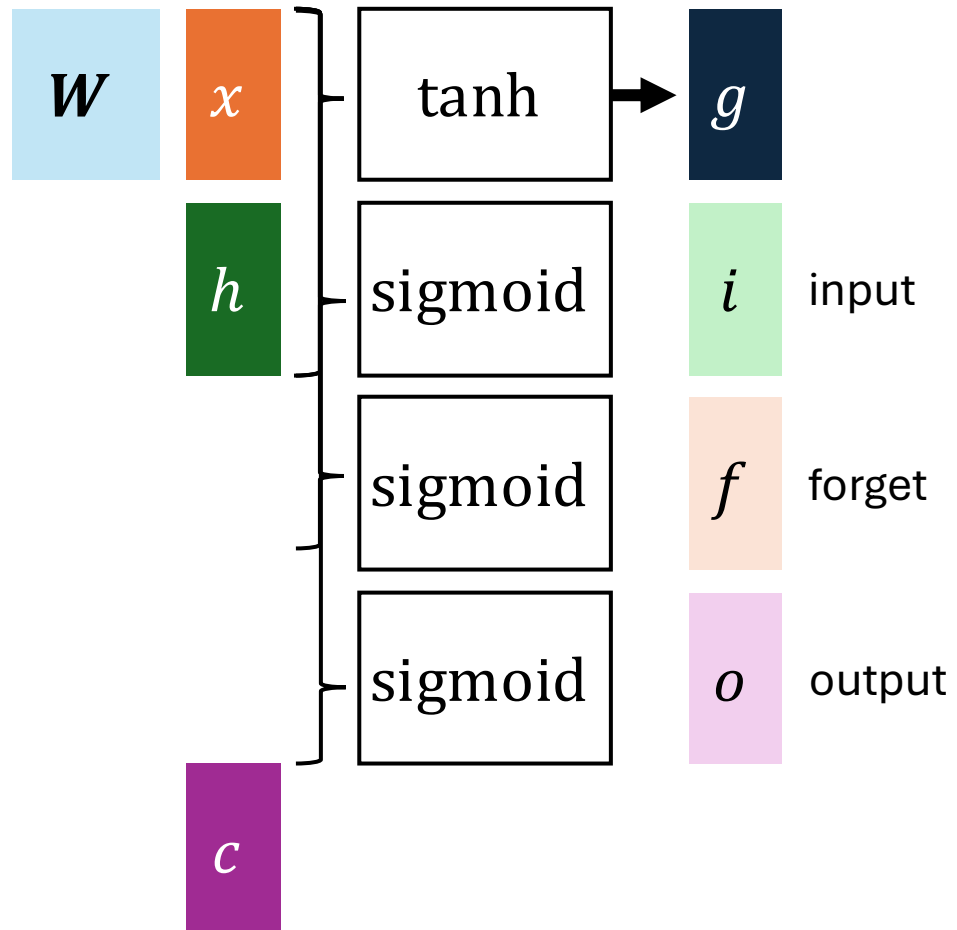


Forwarding/backwarding in time for long sequences = very deep NN
Leads to **vanishing** or **exploding** gradients

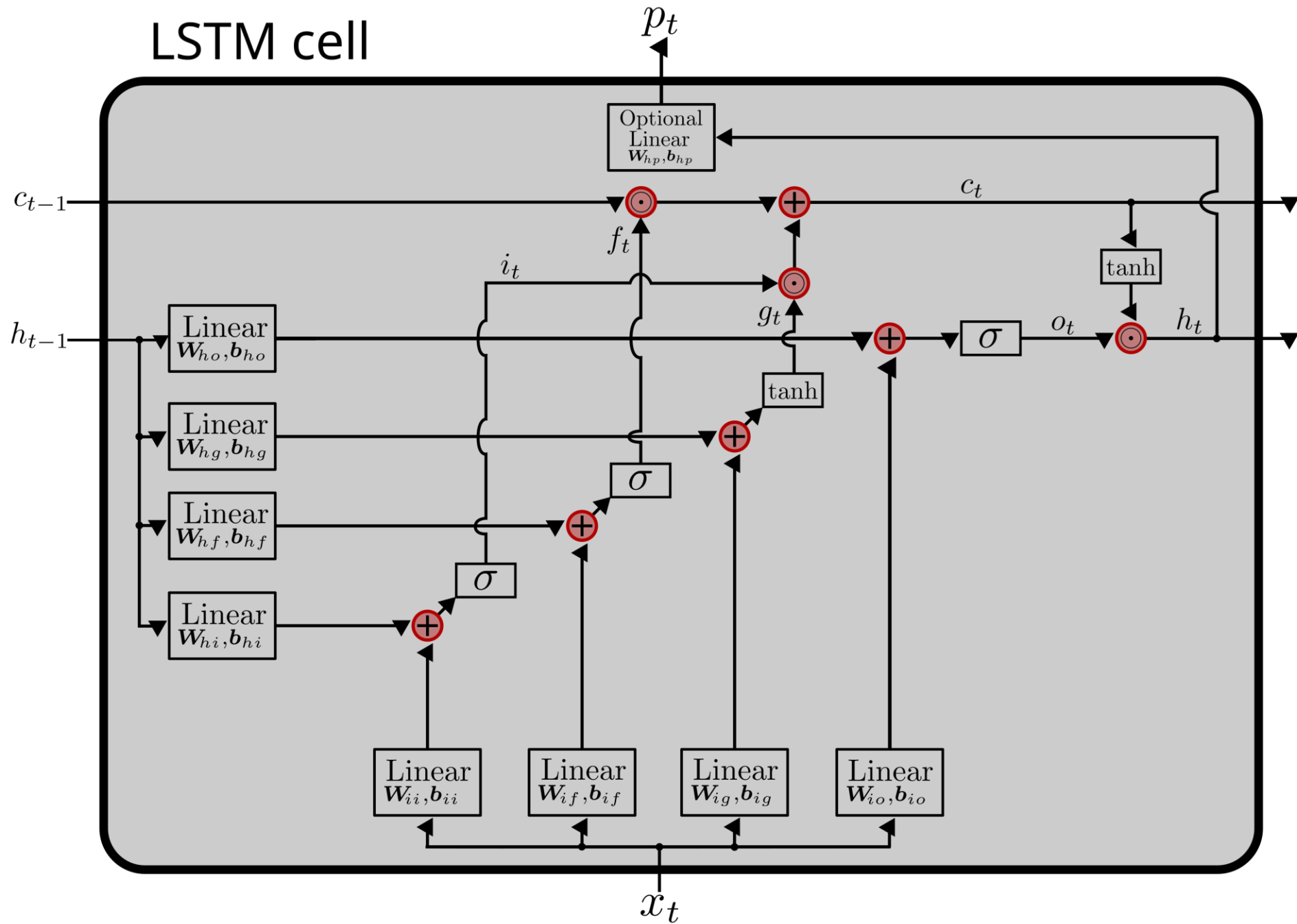
Long Short Term Memory (LSTM)



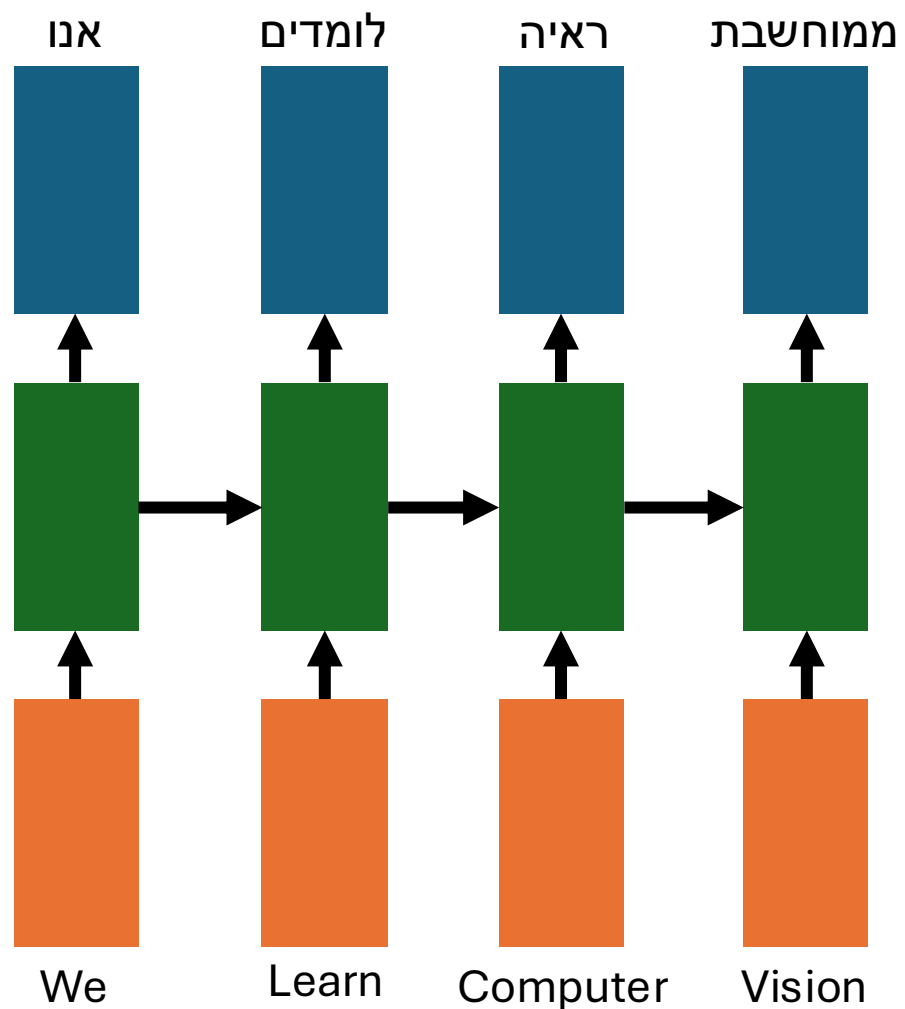
Long Short Term Memory (LSTM)



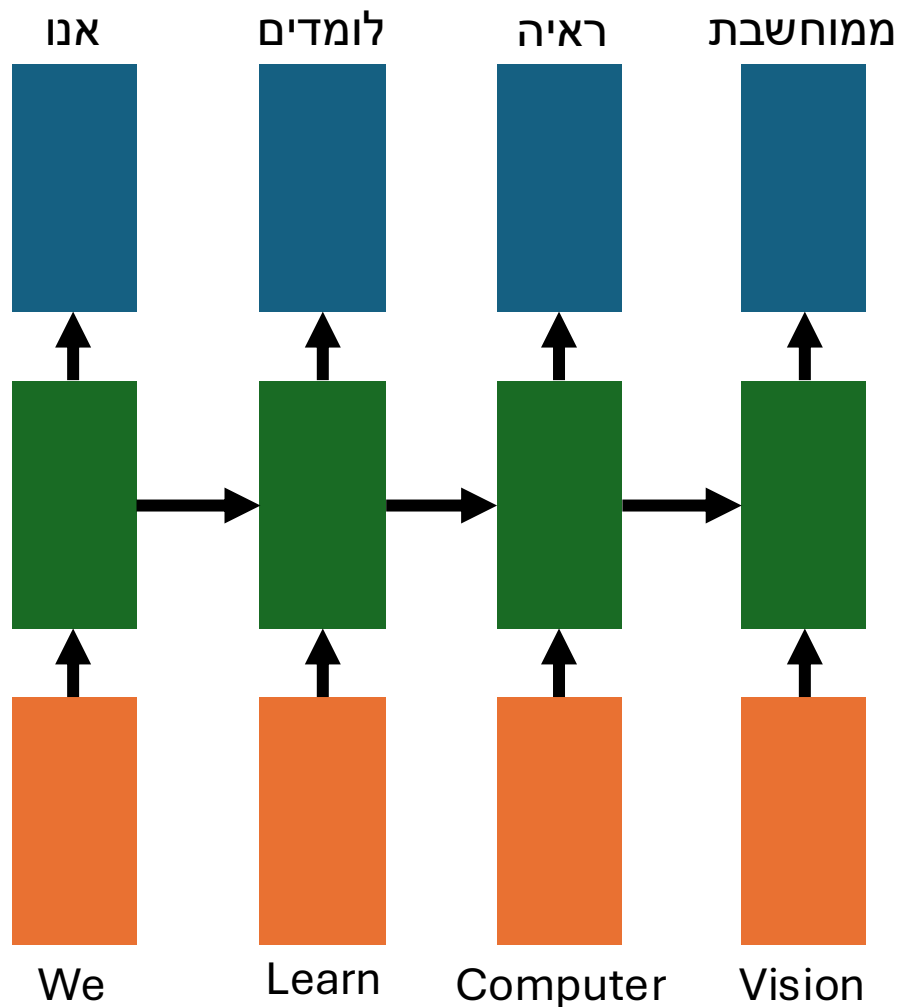
$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$
$$h_t = o_t \odot \tanh(c_t)$$



Sequence to sequence



Sequence to sequence



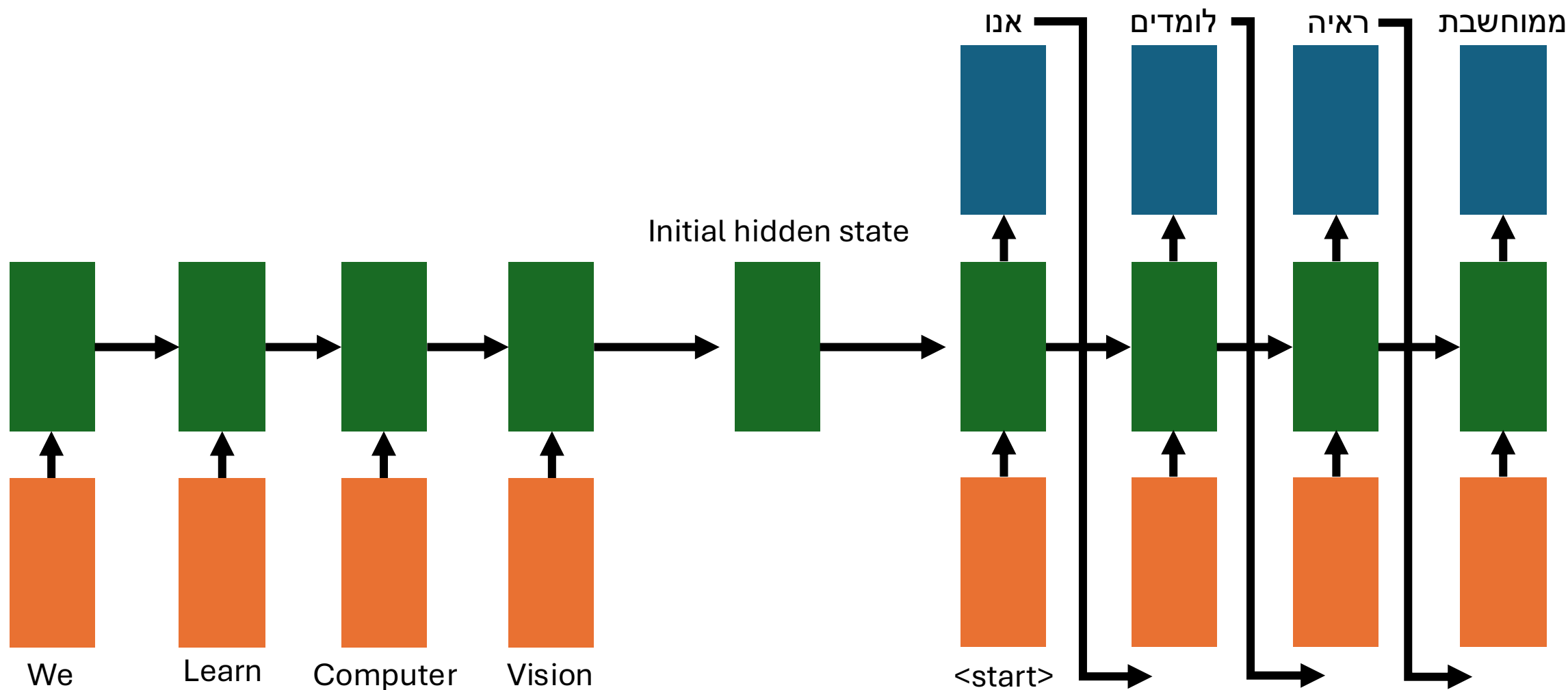
?

We
Learn
Computer
Vision

אנו
לומדים
ממוחשבת
ראיה

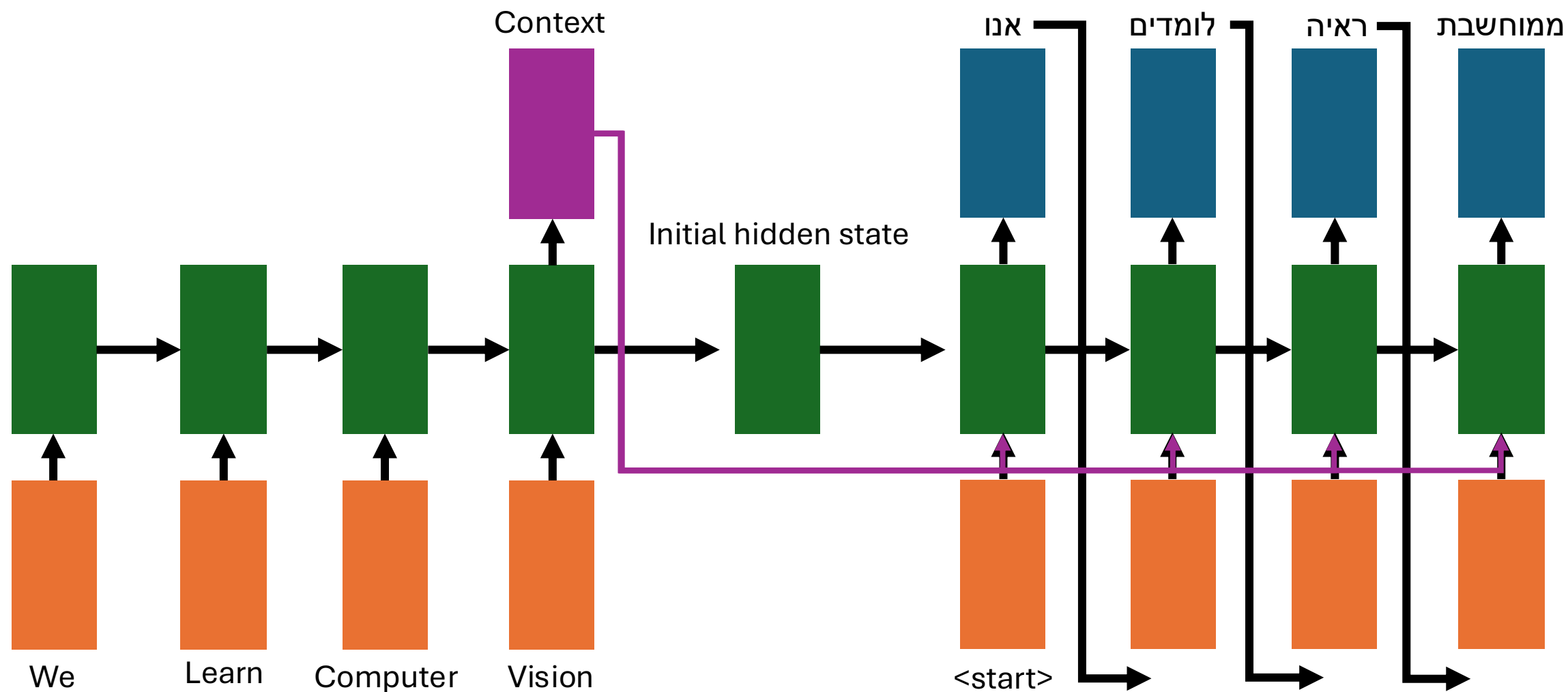
Encoder

Decoder



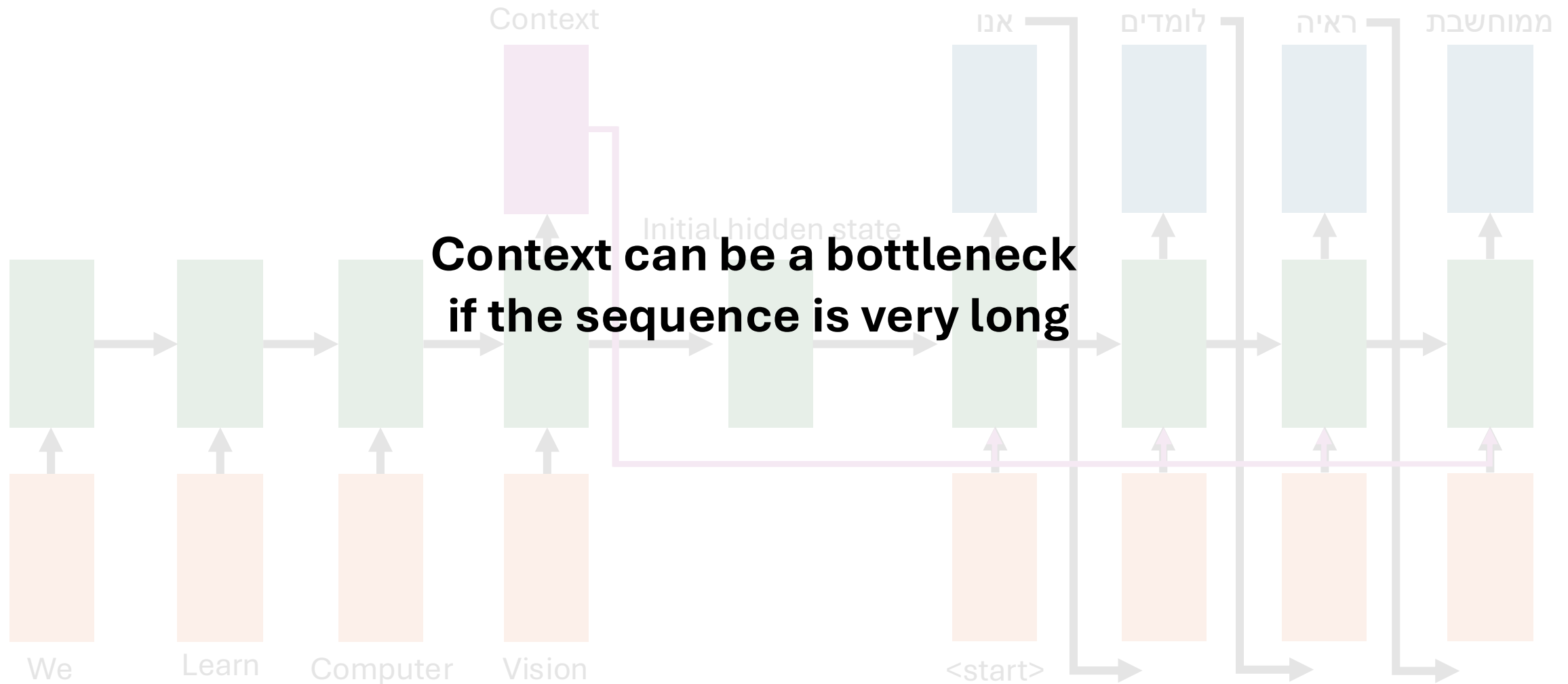
Encoder

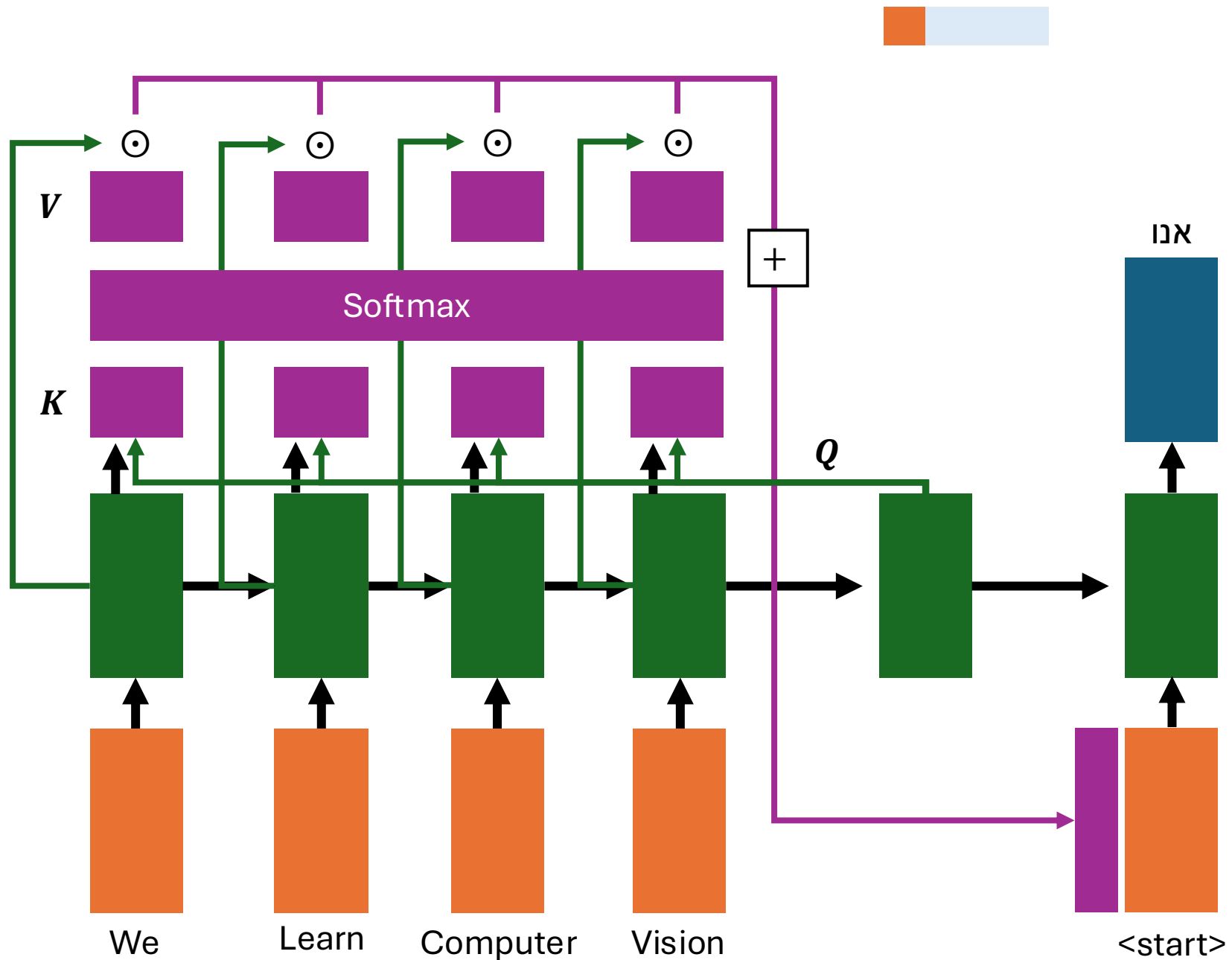
Decoder

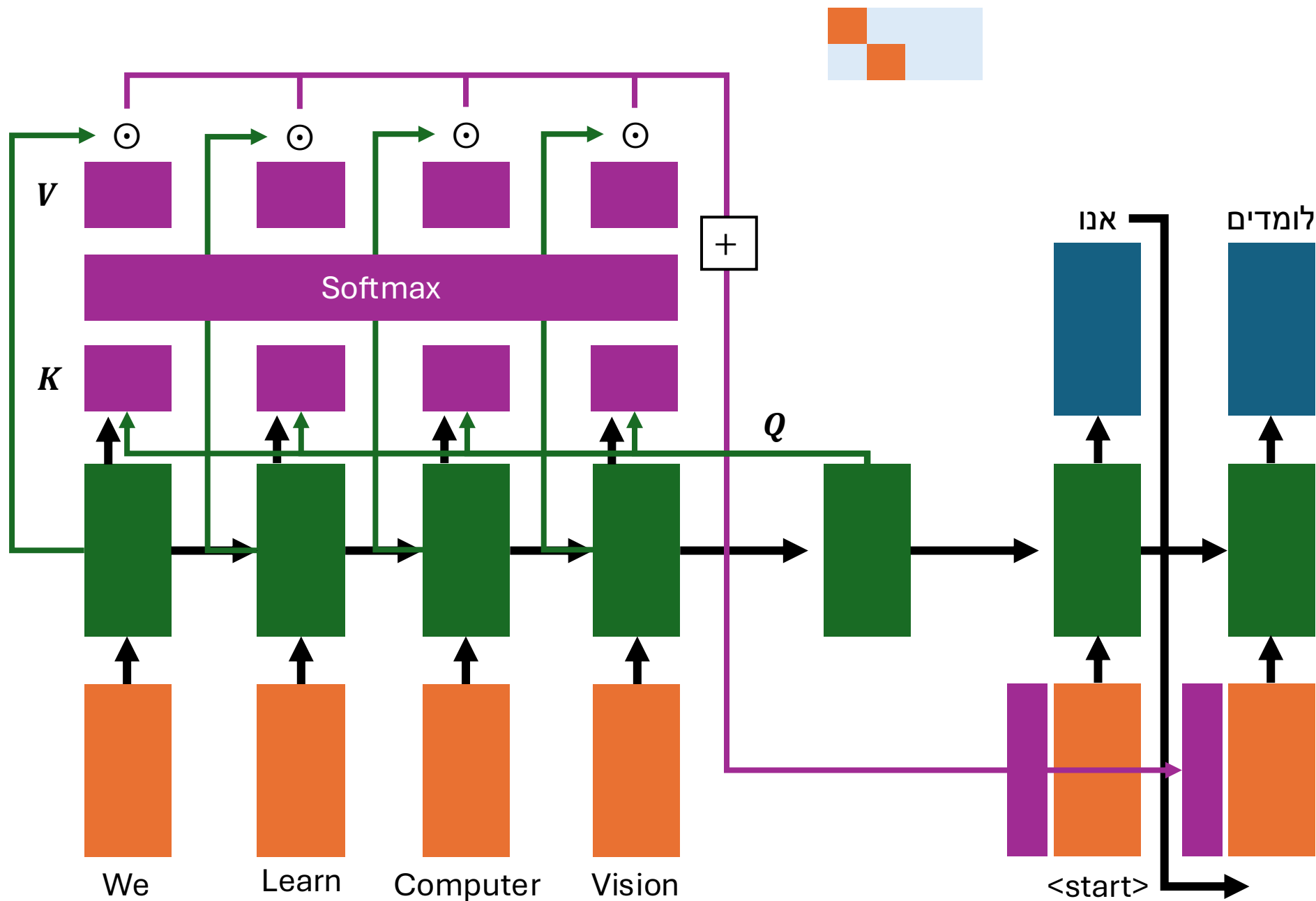


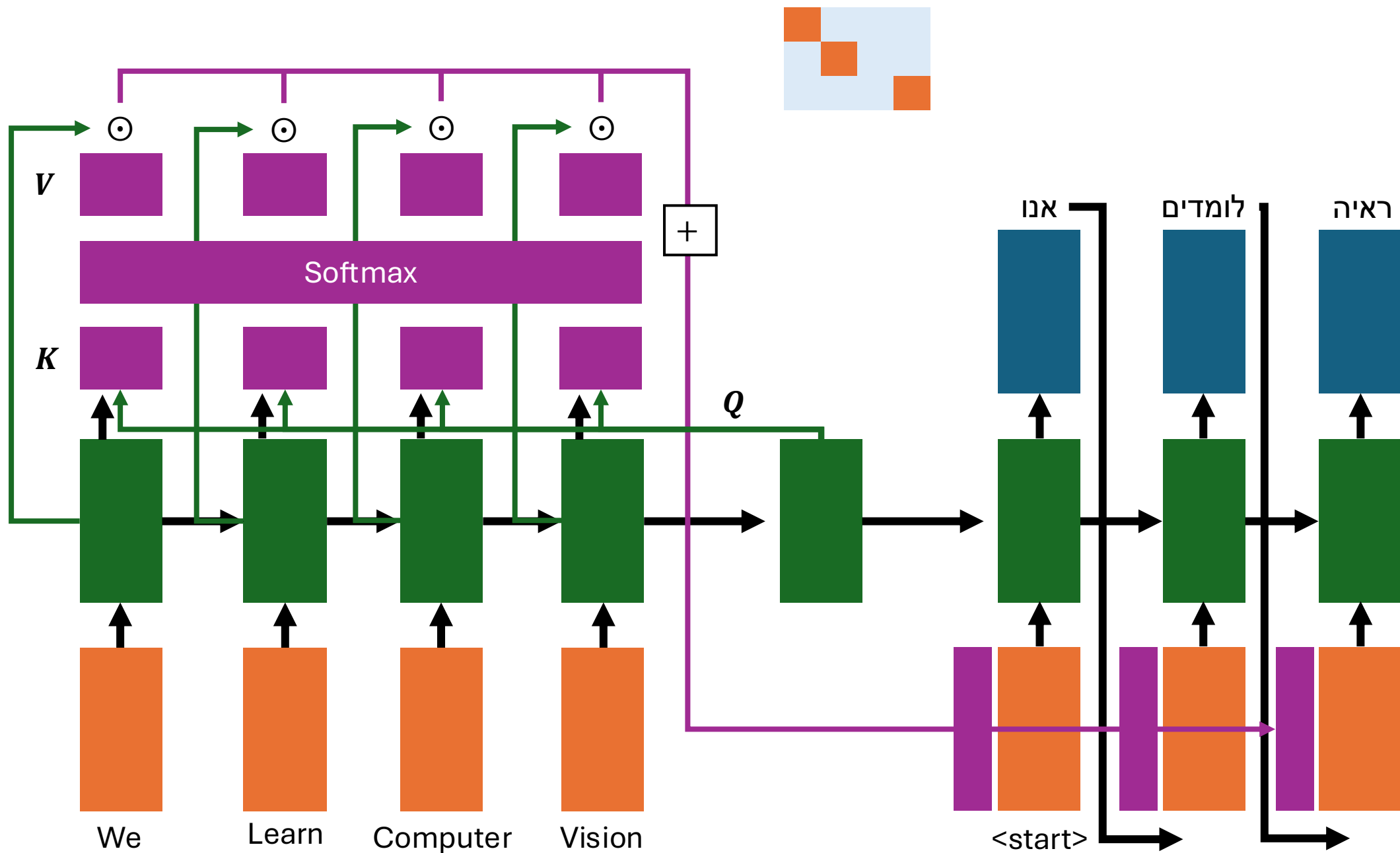
Encoder

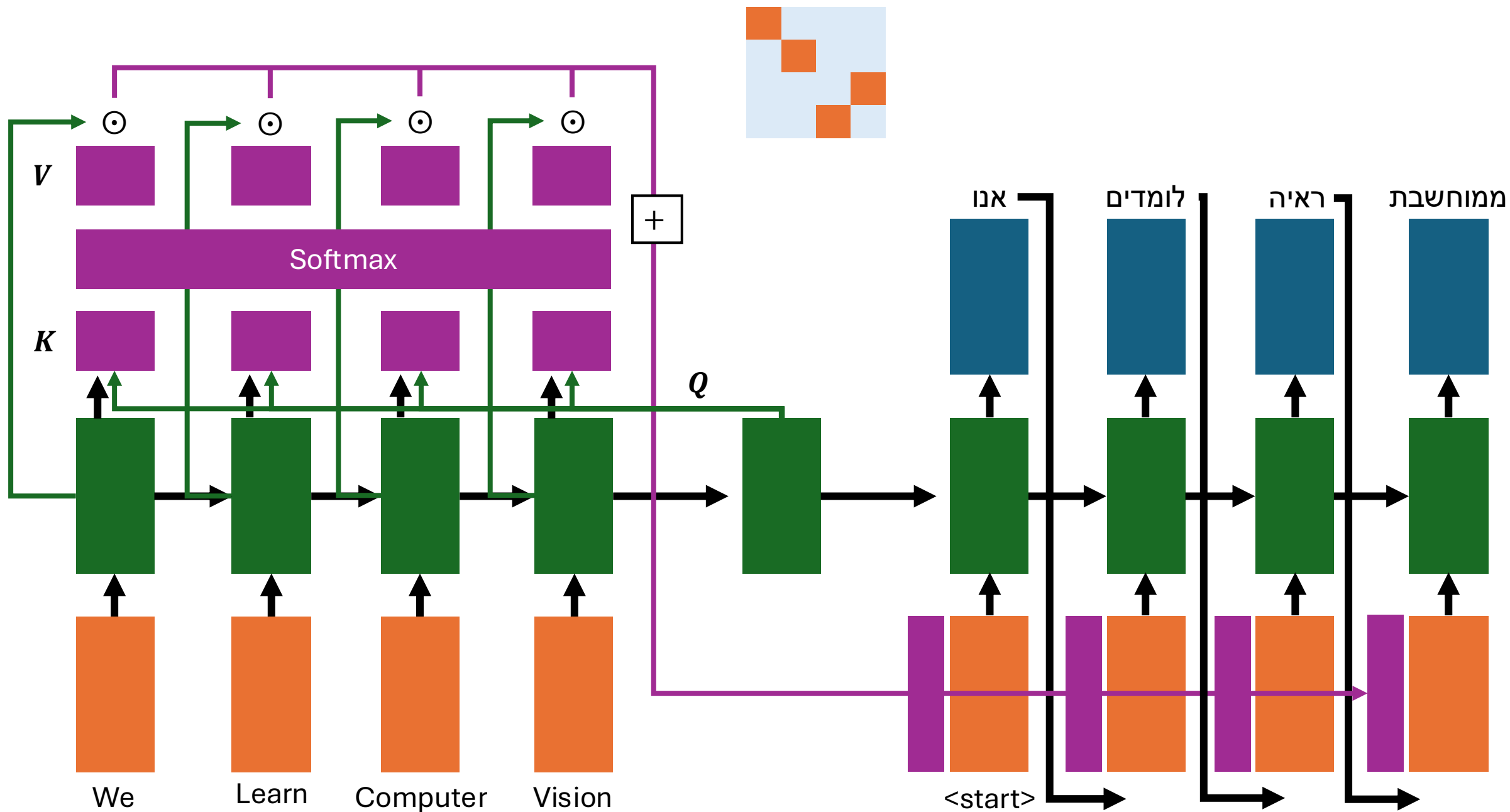
Decoder

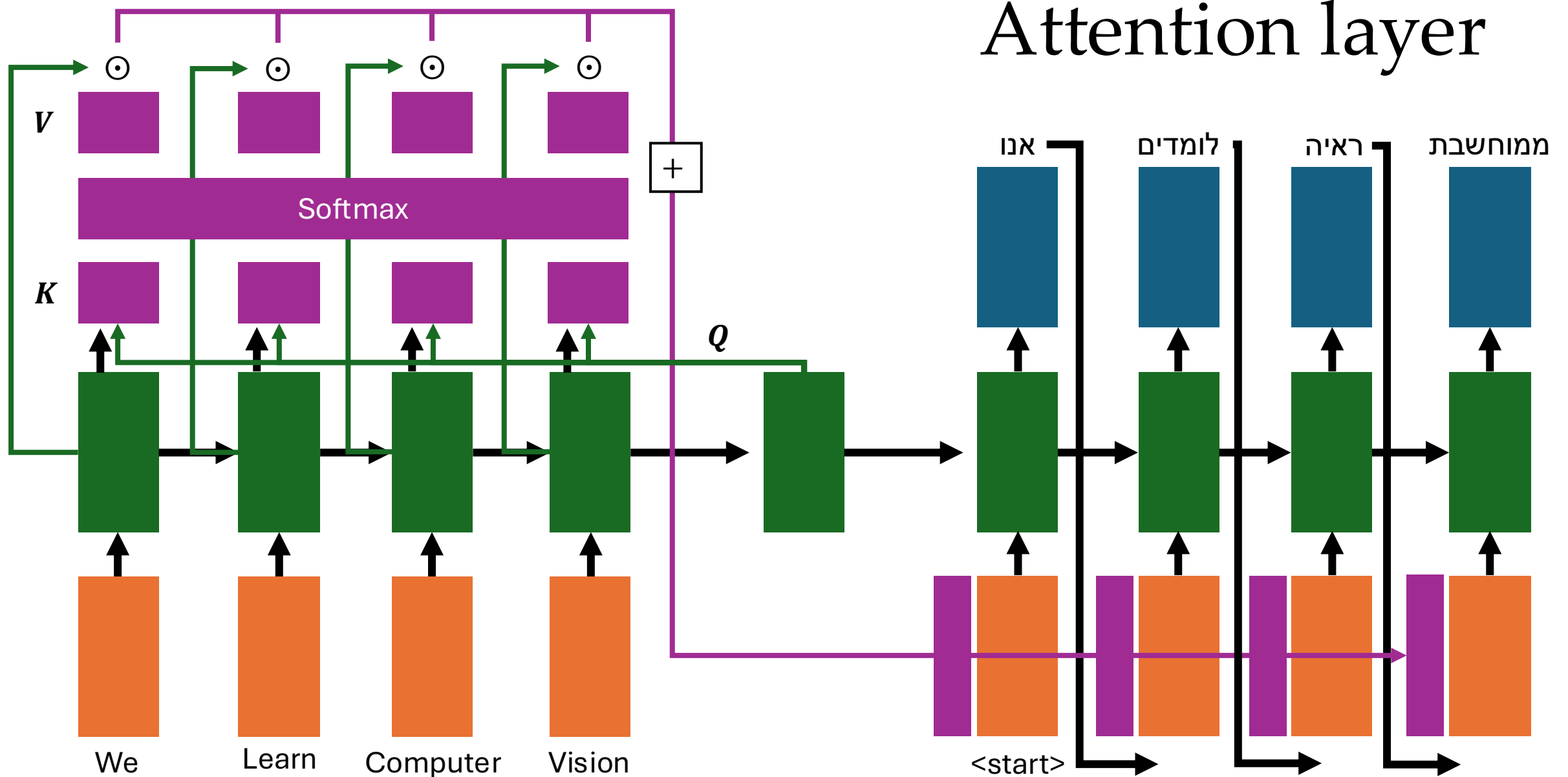












Attention

Inputs:

Query: Q ($N_q \times D_q$)

Input: X ($N_x \times D_x$)

Parameters:

$X \rightarrow K: W_k$ ($D_x \times D_q$)

$X \rightarrow V: W_v$ ($D_x \times D_v$)

Compute:

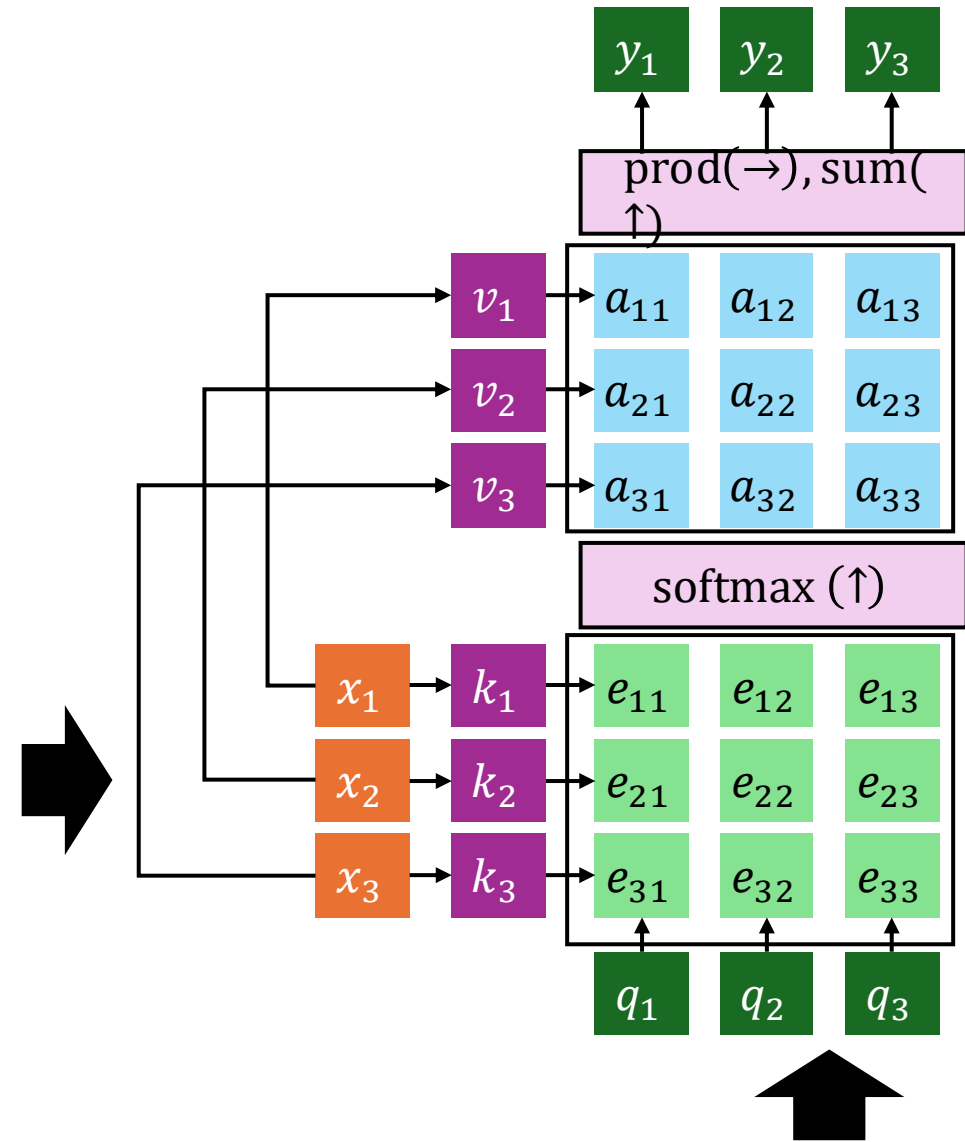
Keys: $K = XW_k$ ($N_x \times D_q$)

Values: $V = XW_v$ ($N_x \times D_v$)

Similarities: $E = \frac{QK^T}{\sqrt{D_q}}$ ($N_q \times N_x$)

Attention: $A = \text{softmax}(E)$ ($N_q \times N_x$)

Outputs: $Y = AV$ ($N_q \times D_v$)



Self attention

Inputs:

Input: $X (N_x \times D_x)$

Parameters:

$X \rightarrow Q: W_q (D_x \times D_q)$

$X \rightarrow K: W_k (D_x \times D_q)$

$X \rightarrow V: W_v (D_x \times D_v)$

Compute:

Query: $Q = XW_q (N_x \times D_q)$

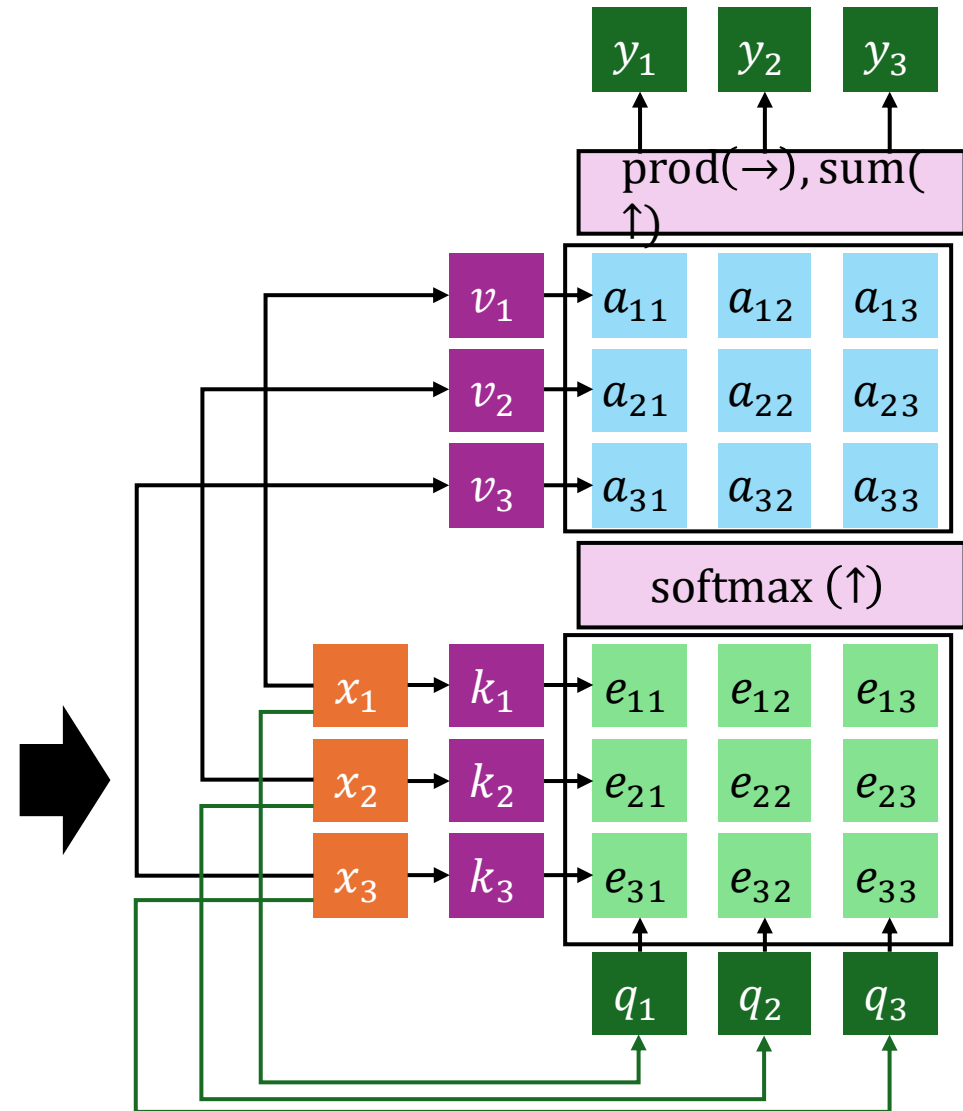
Keys: $K = XW_k (N_x \times D_q)$

Values: $V = XW_v (N_x \times D_v)$

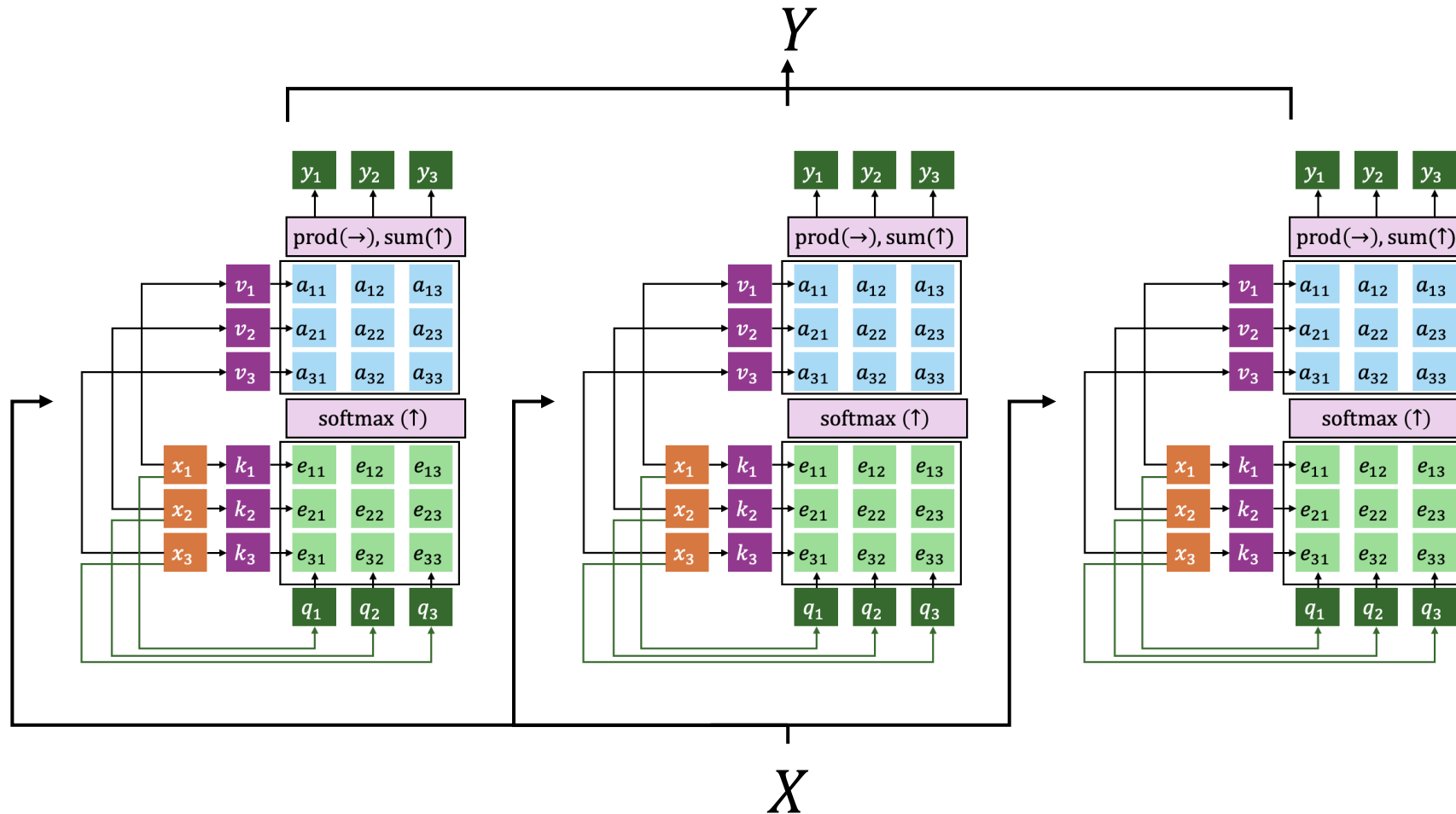
Similarities: $E = \frac{QK^T}{\sqrt{D_q}} (N_q \times N_x)$

Attention: $A = \text{softmax}(E) (N_q \times N_x)$

Outputs: $Y = AV (N_q \times D_v)$

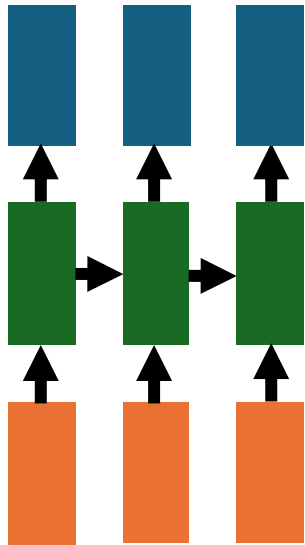


Multi head self attention



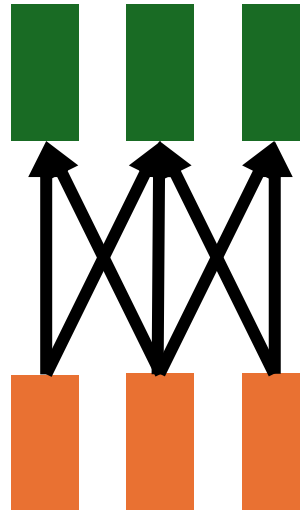
Three ways to process sequences

RNN



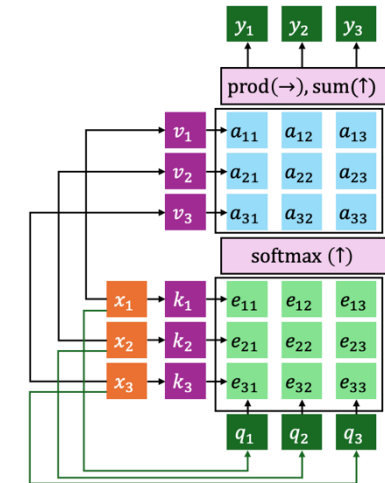
- + Large and adjustable receptive field (via hidden state)
- Not parallelizable: sequential processing

CNN



- + Highly parallelizable
- Fixed receptive field, demands many layers

Attention



- + Receptive field = entire sequence
- + Parallelizable
- Very memory intensive

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

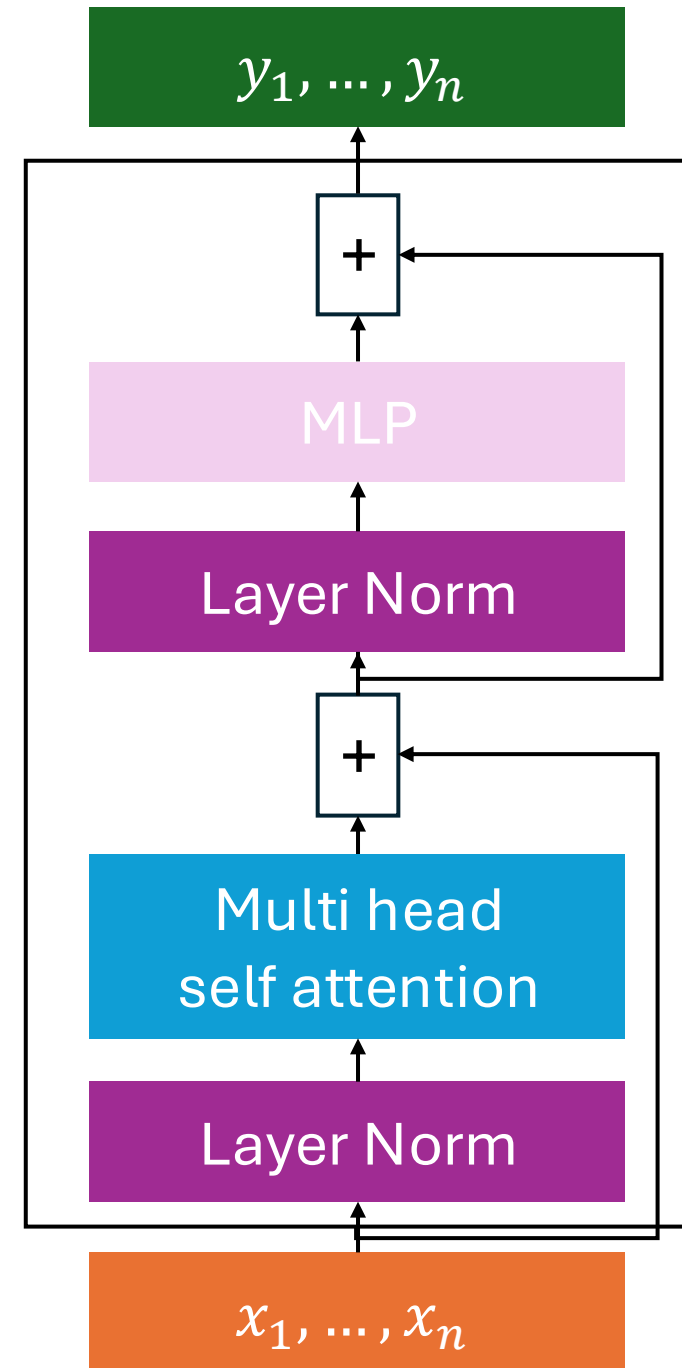
Transformer layer

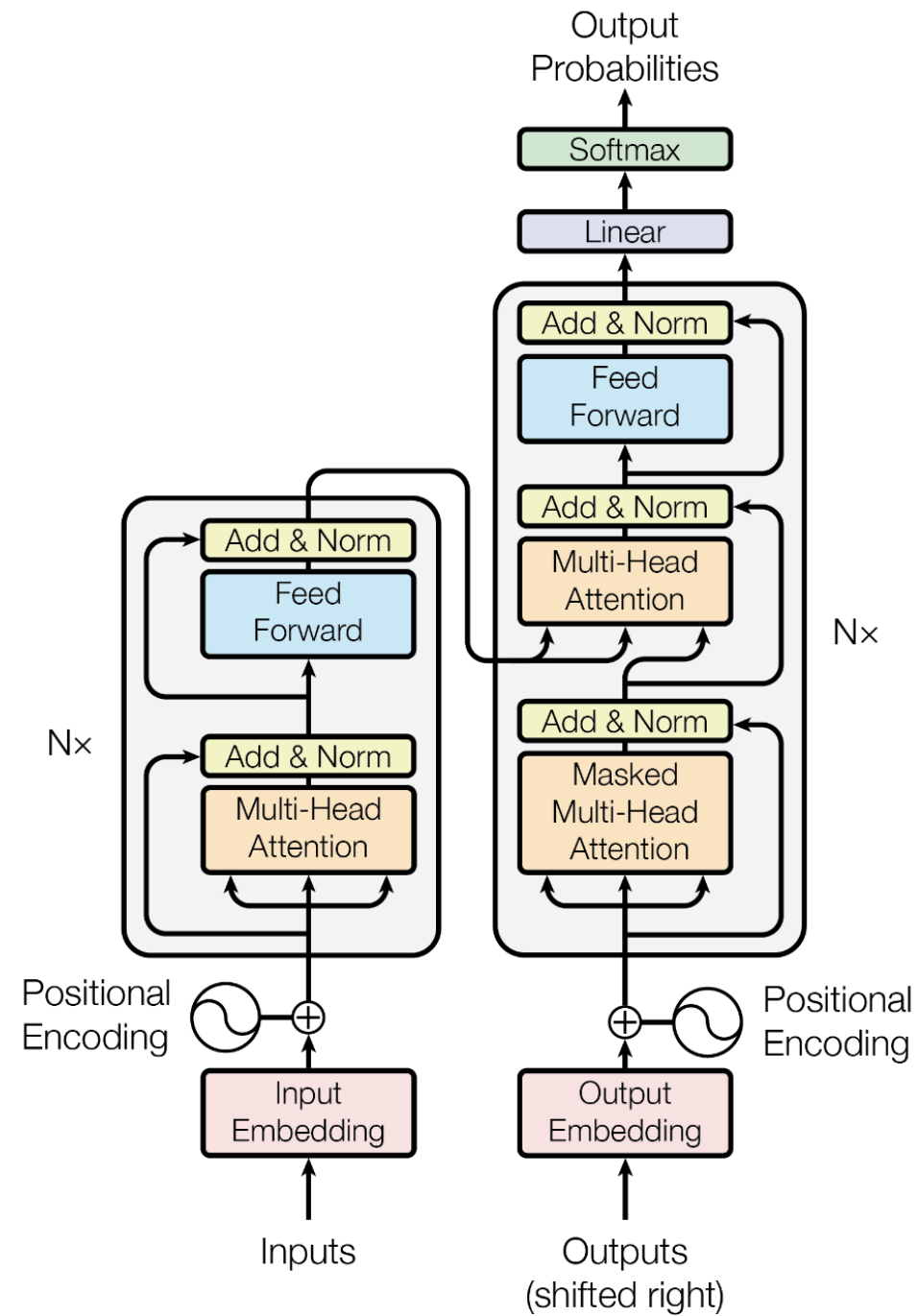
Inputs: $x_1, \dots, x_n$ (n tokens in D dimensions)

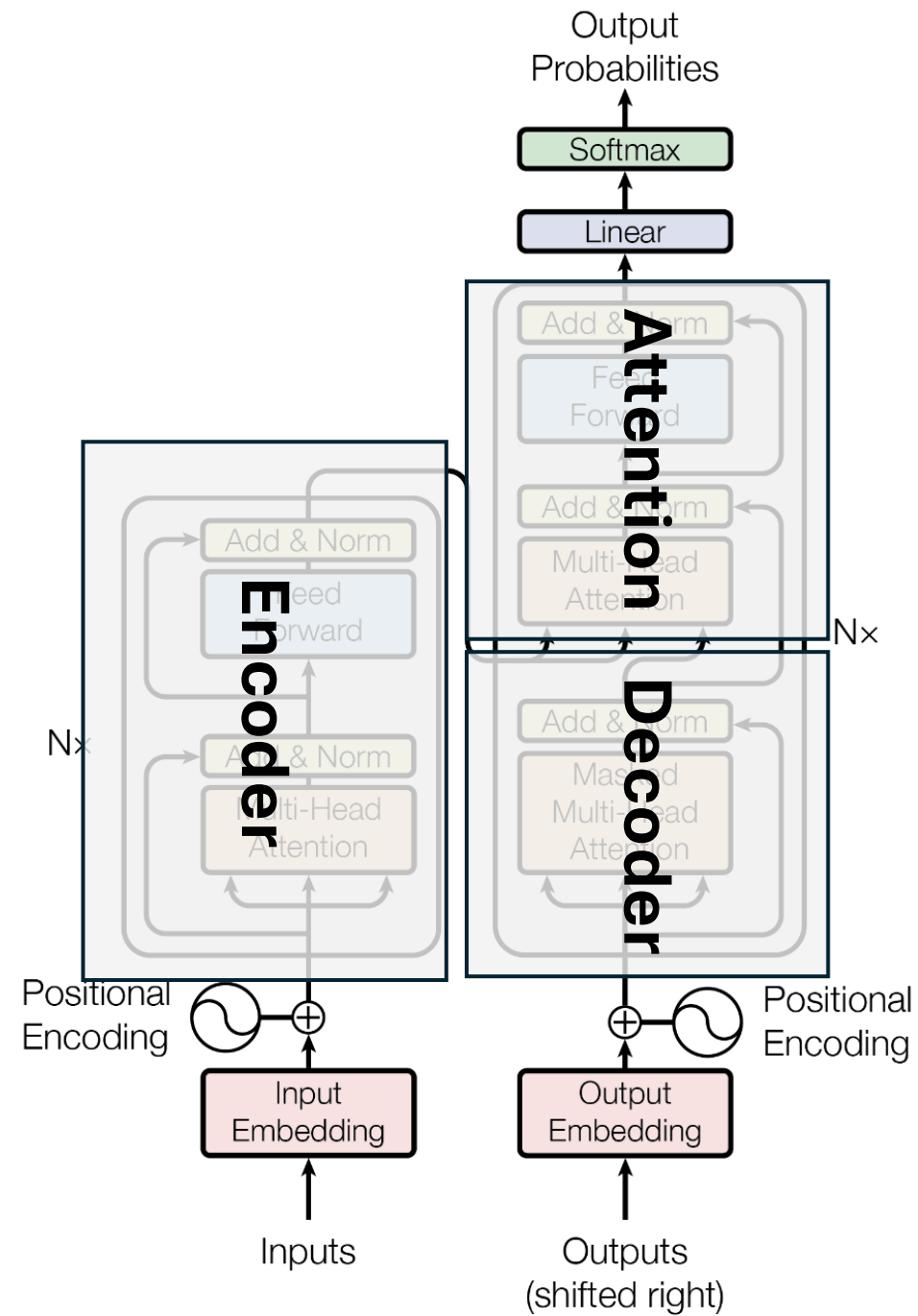
Outputs: $y_1, \dots, y_n$ (n tokens in D dimensions)

Highly scalable

Highly parallelizable







Transformers network

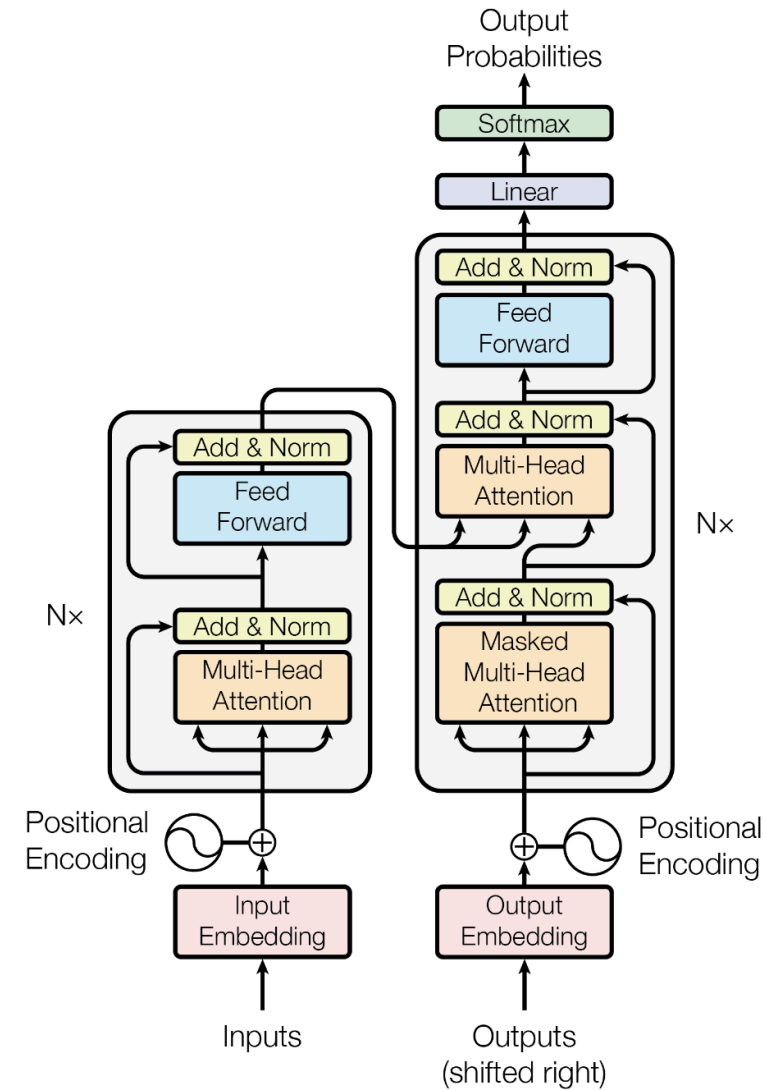
Pretraining:

Consume A LOT of data

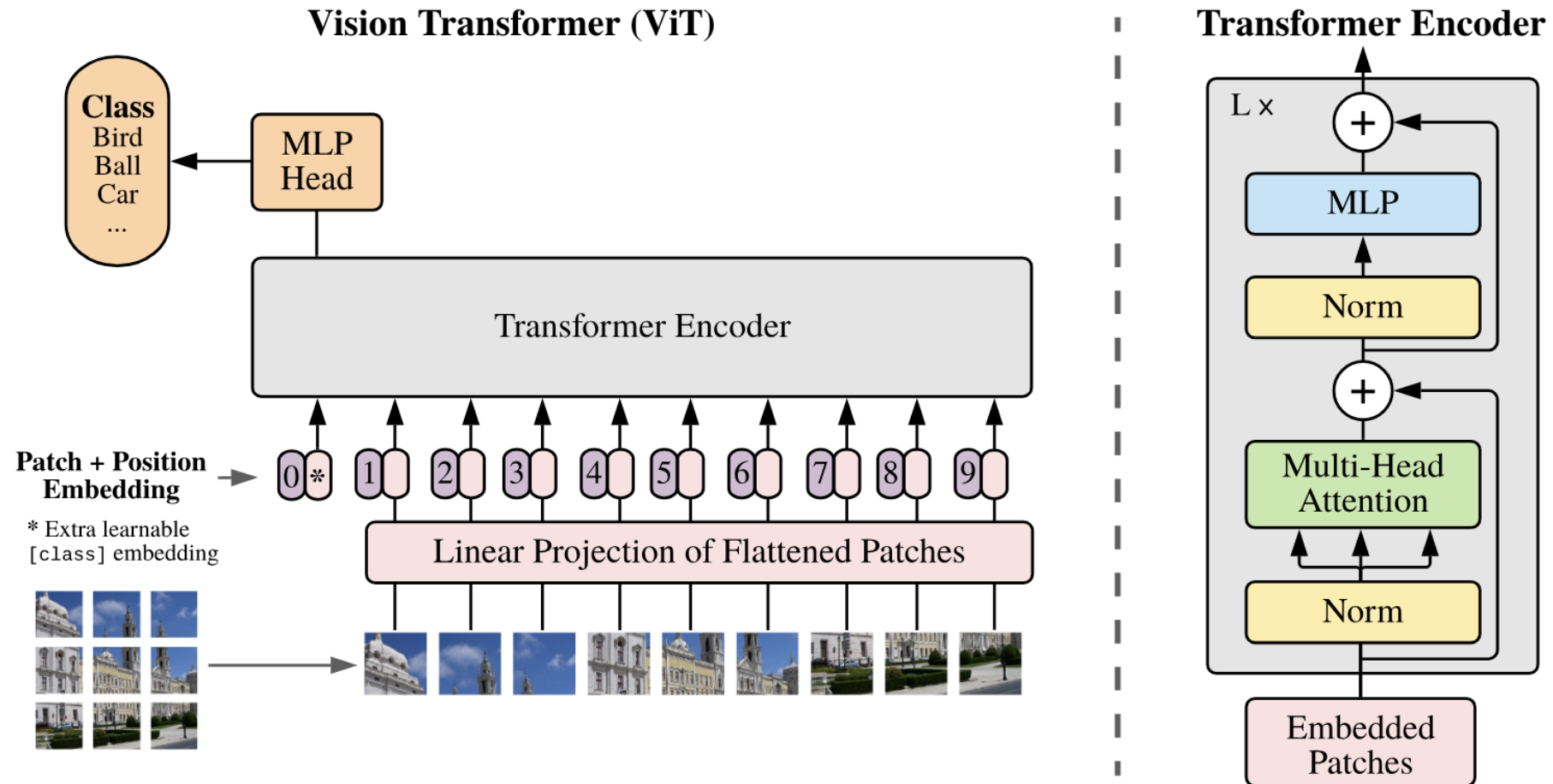
Train using self-supervision

Finetuning:

Train on specific data for a specific task

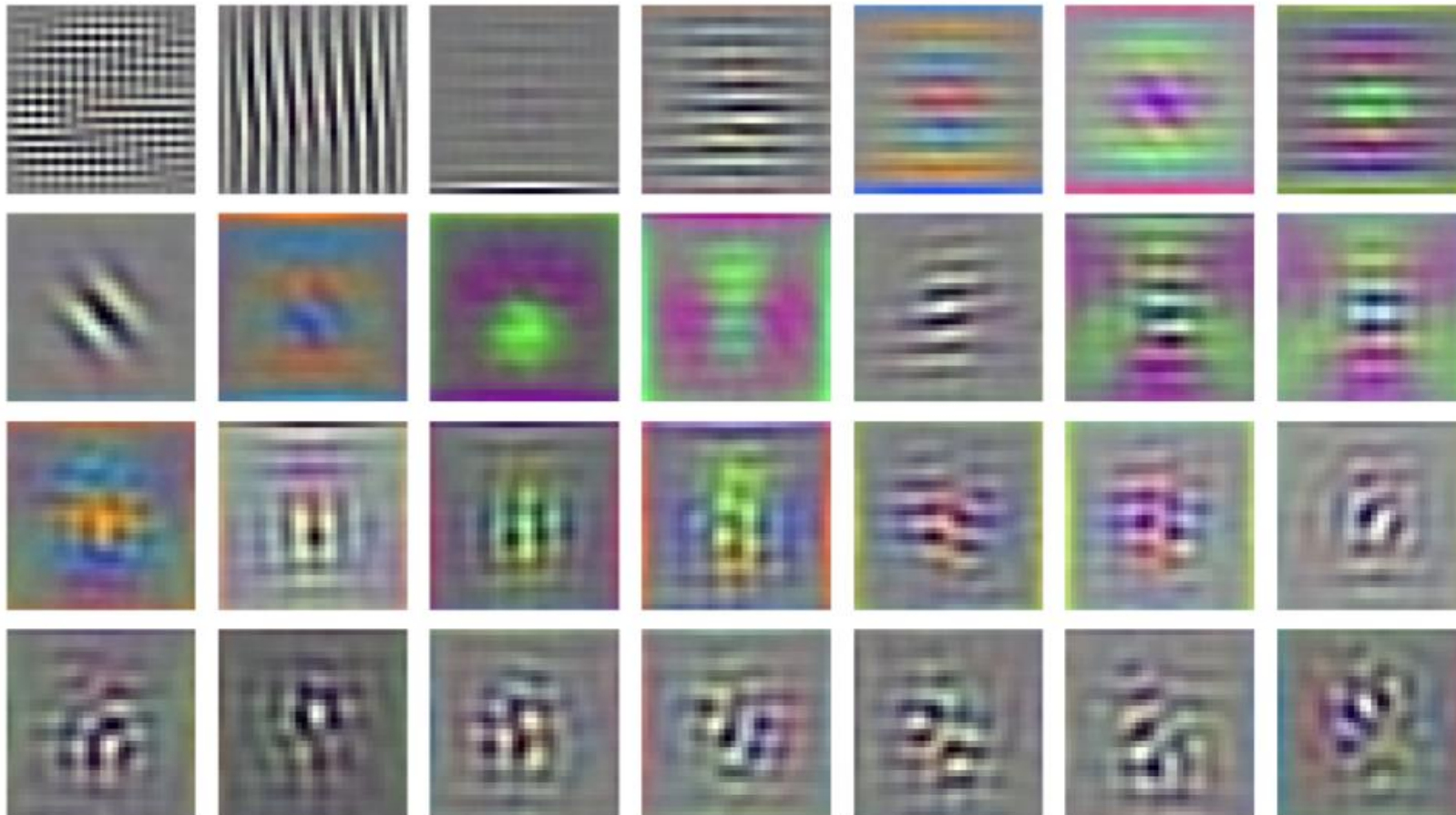


Vision transformer (ViT)



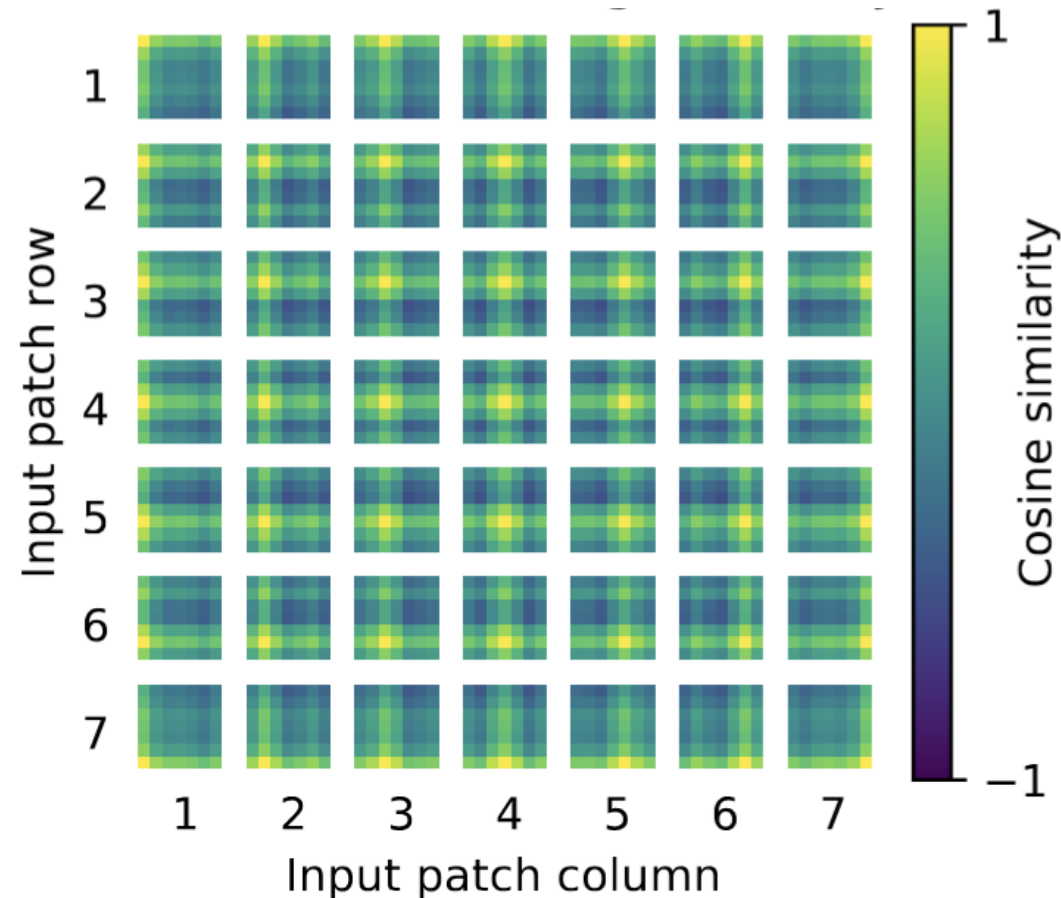
Vision transformer (ViT)

Examples of embedding filters (weights representations in RGB):



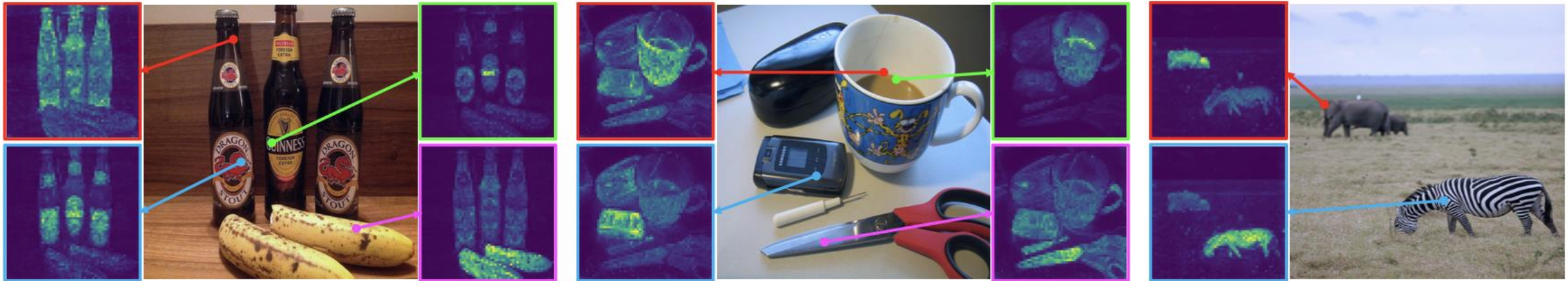
Vision transformer (ViT)

Similarity of embedding of positional encoding:



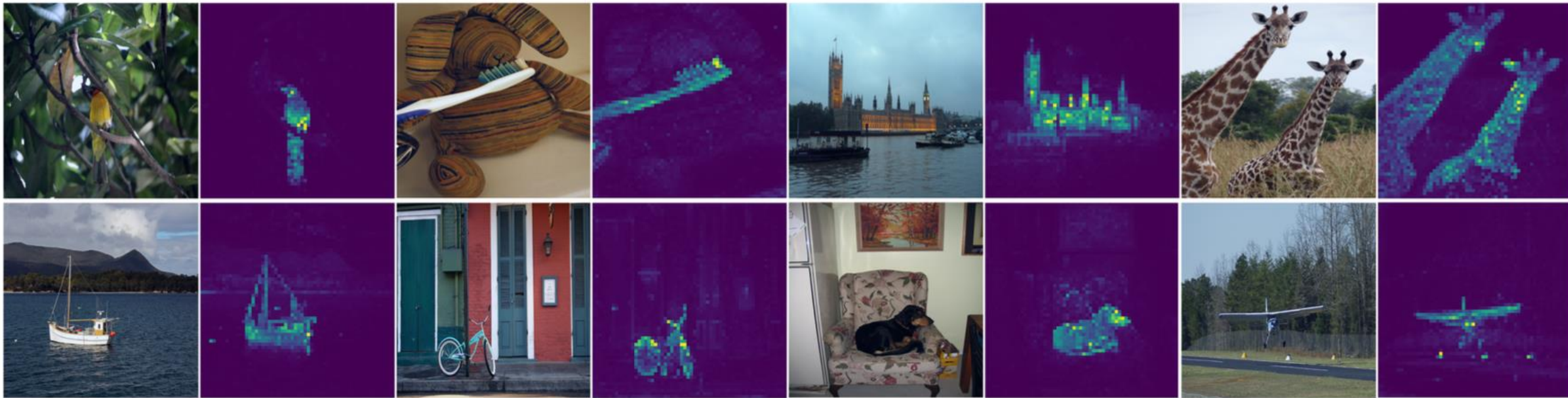
Vision transformer (ViT)

Attention maps for pixel patches:



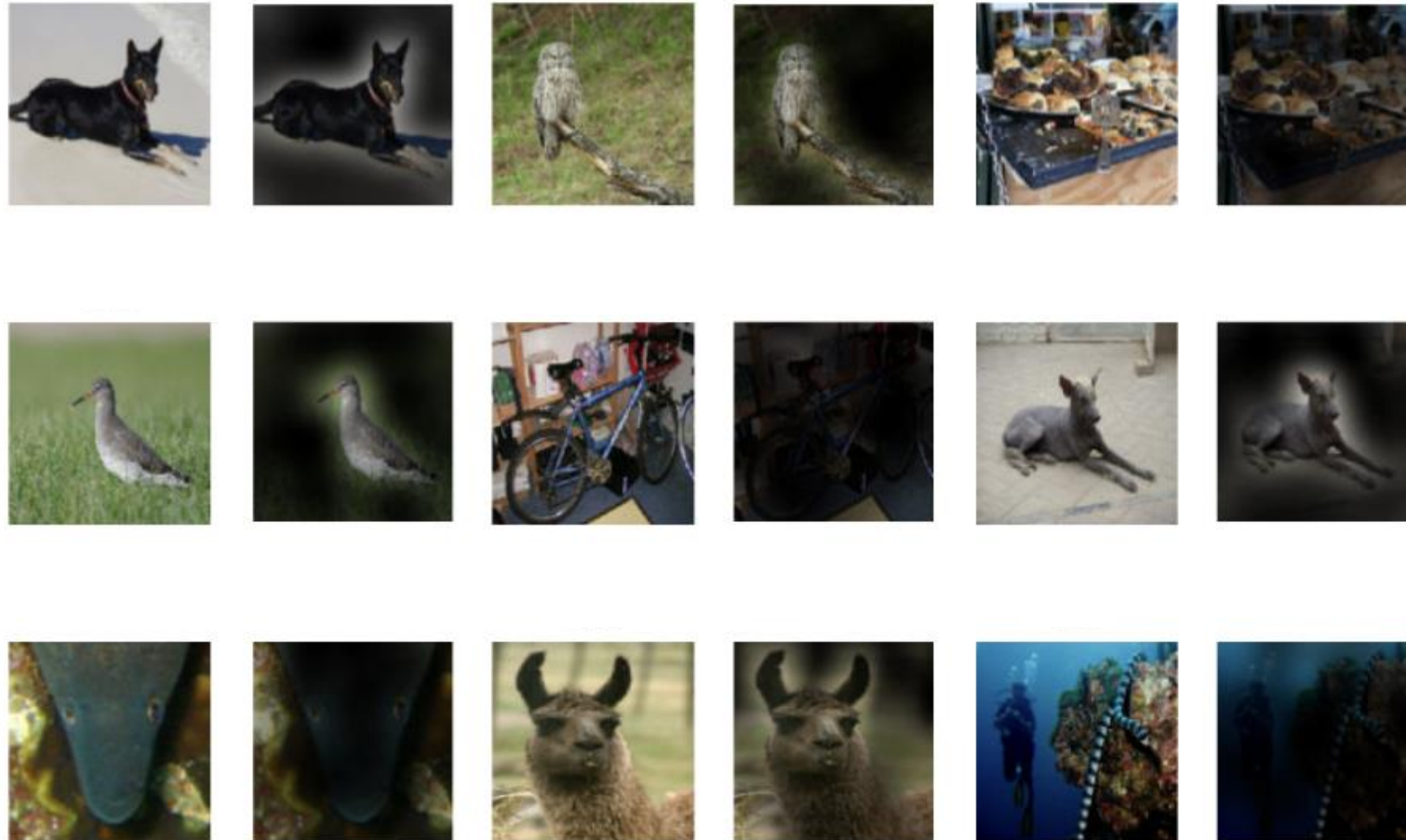
Vision transformer (ViT)

Attention maps for [cls] token:



Vision transformer (ViT)

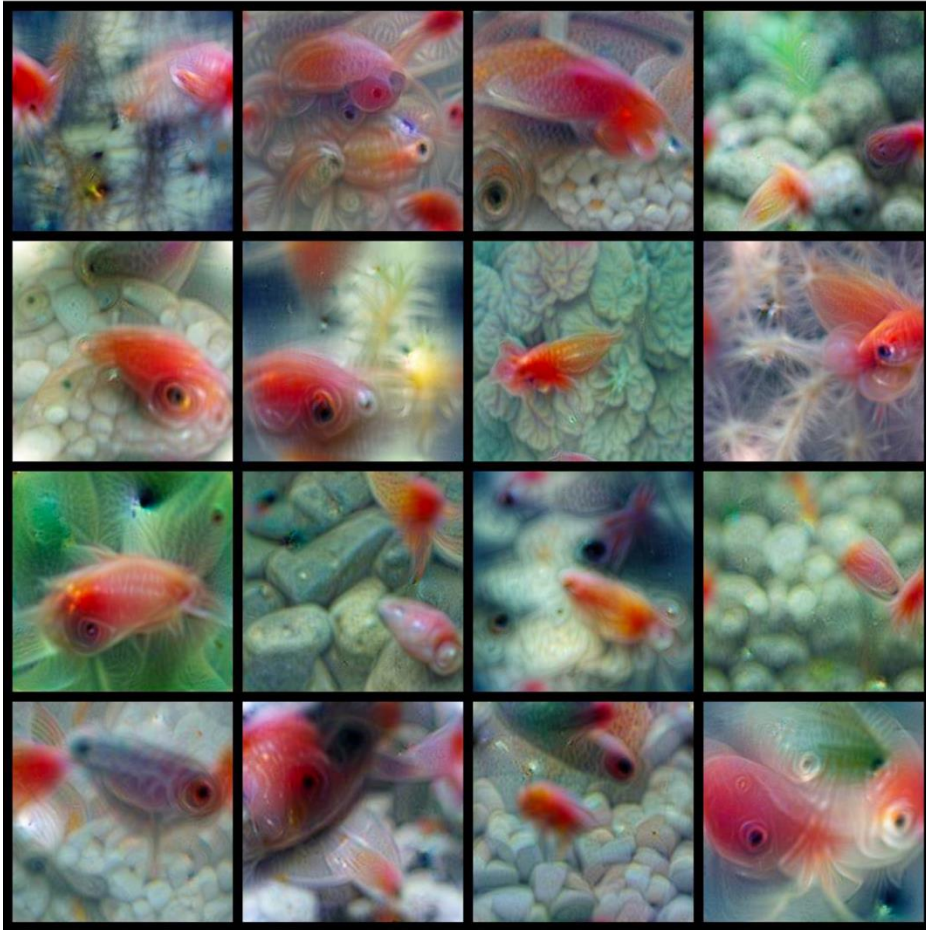
Attention maps for [CLS] token:



Vision transformer (ViT)

Image optimization with respect to logits:

Goldfish



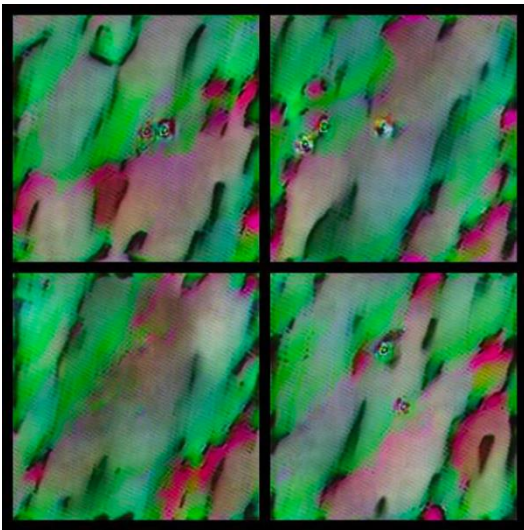
Bald Eagle



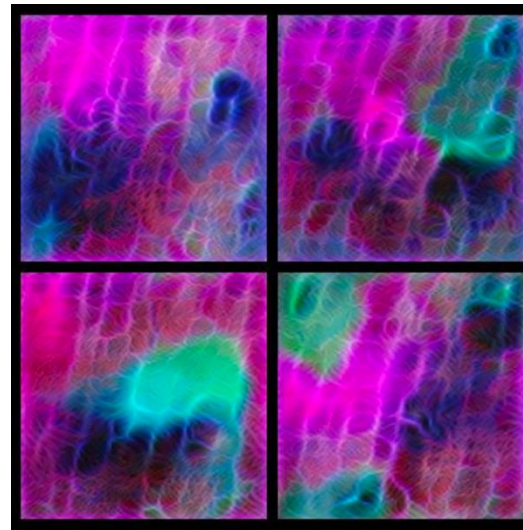
Vision transformer (ViT)

Image optimization with respect to [CLS] token:

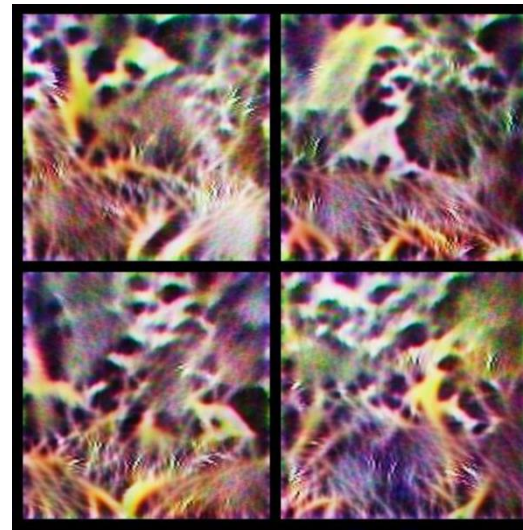
Layer 1



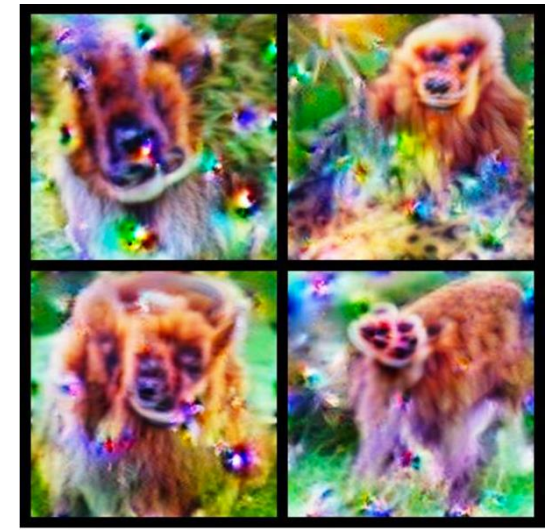
Layer 3



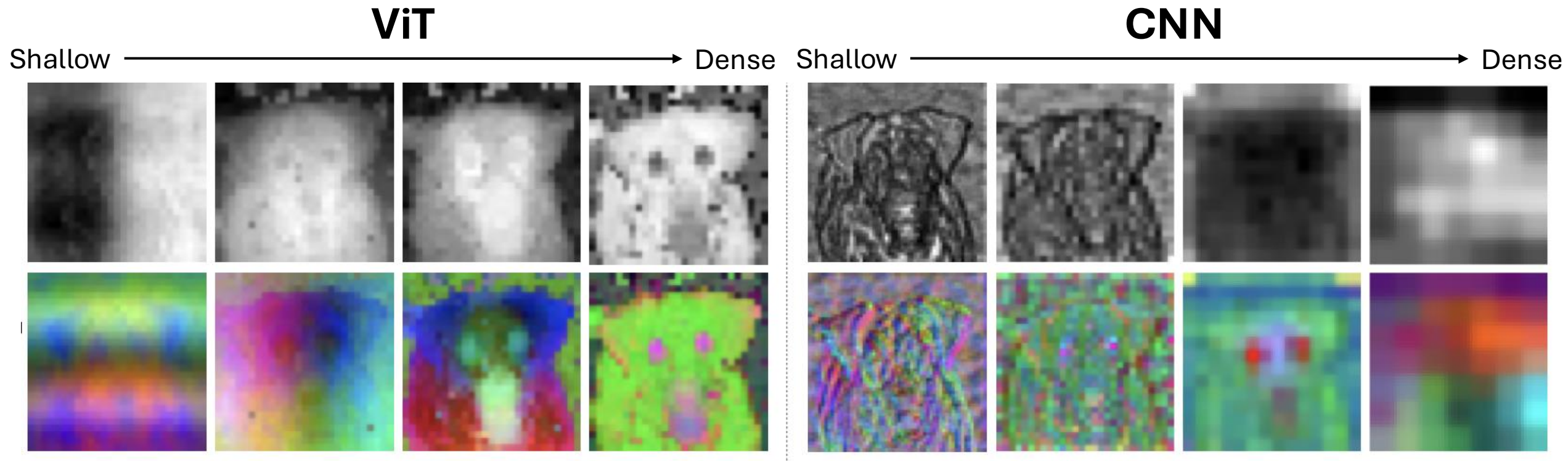
Layer 5



Layer 10

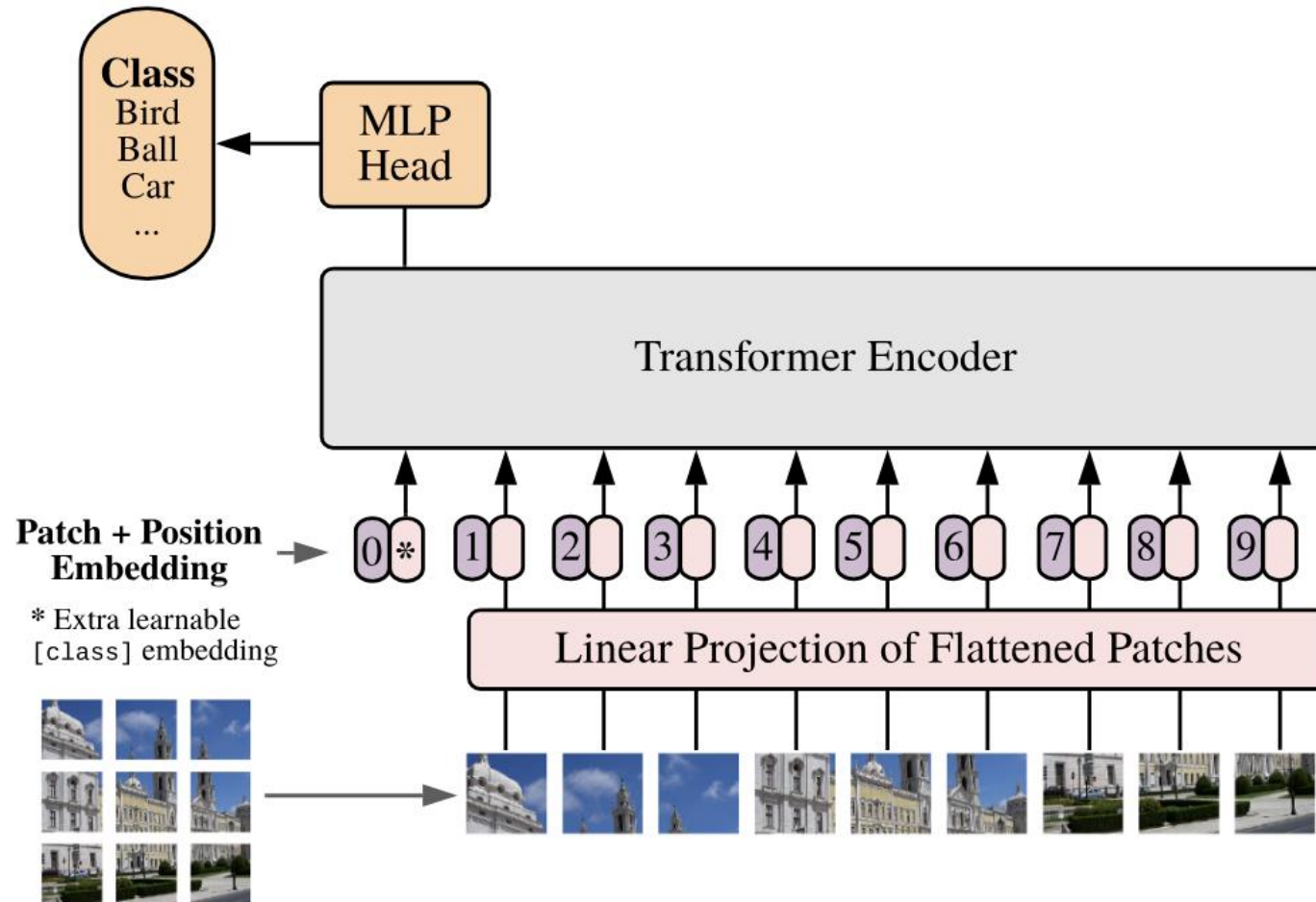


Vision transformer (ViT)



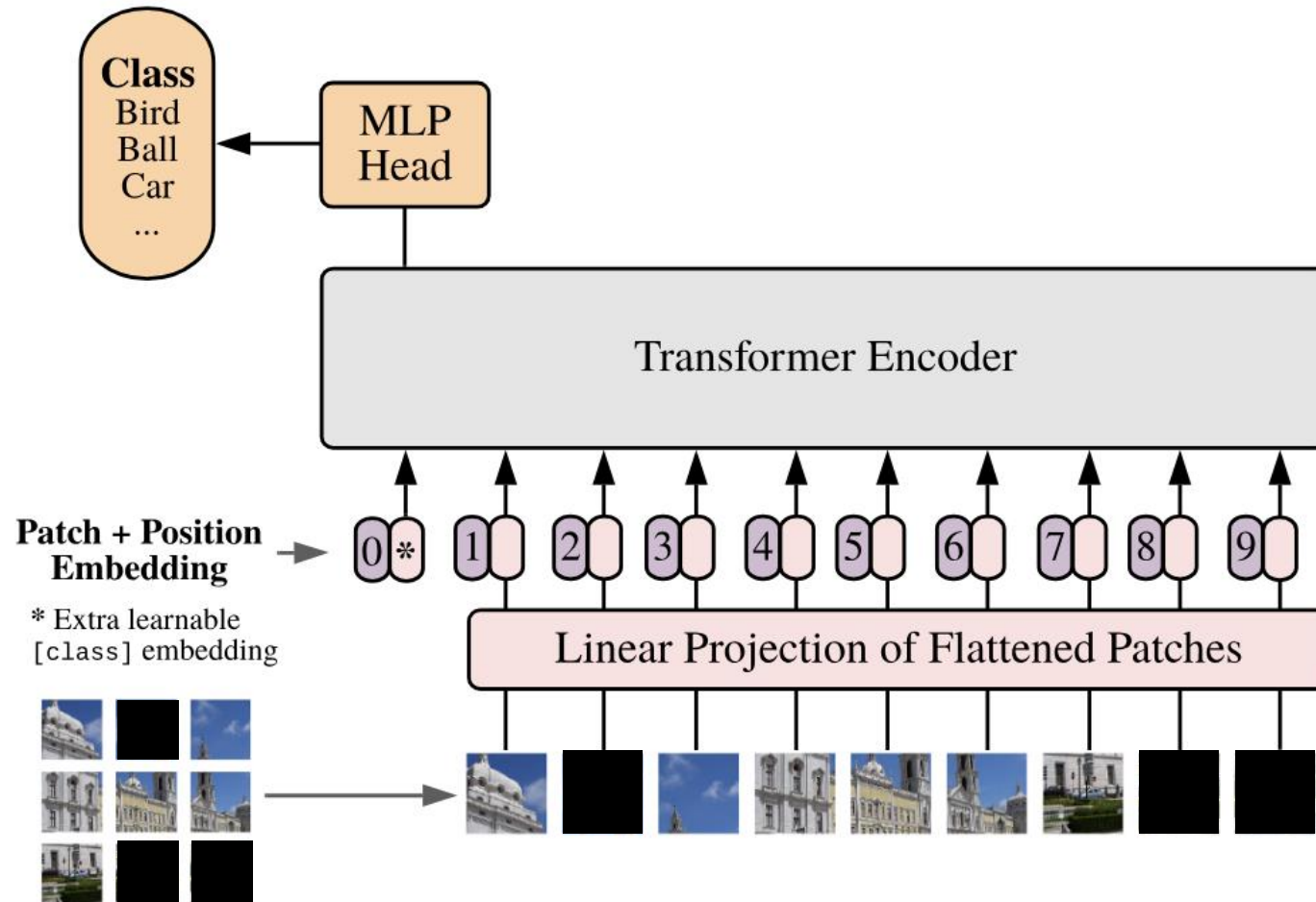
Vision transformer (ViT)

Robustness:



Vision transformer (ViT)

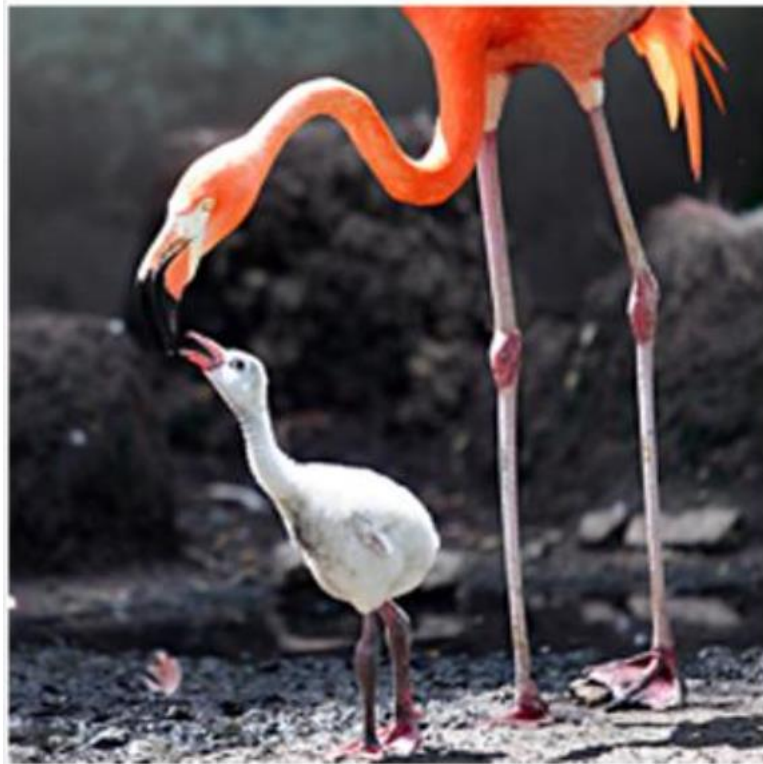
Robustness:



Vision transformer (ViT)

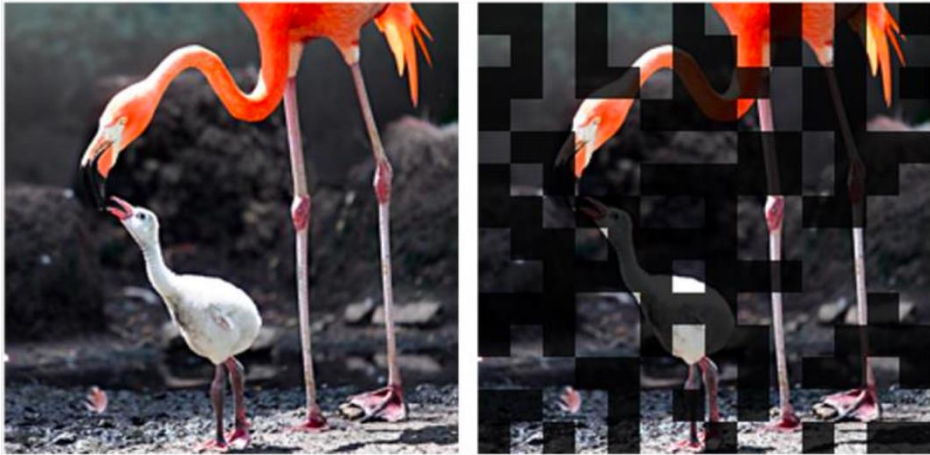
Robustness:

Random PatchDrop



Vision transformer (ViT)

Robustness:



Random PatchDrop:

- Efficiency
- Augmentation
- Showing redundancy

Vision transformer (ViT)

Other properties

- Benefits from larger train sets
- Relative robustness to adversarial examples
- Shape/texture may conflict



Complexity

CNN:

$$y_i = \sum_j x_{i+j} w_j$$

Self attention:

$$y_i = \sum_j \sigma(q_i k_j) v_j$$

Complexity

CNN:

$$y_i = \sum_j x_{i+j} w_j$$

$O(n)$

Self attention:

$$y_i = \sum_j \sigma(q_i k_j) v_j$$

$O(n^2)$

Efficient transformers

